

SP-2: Lava API Migration to Go (Gin + HTTP/3 + Brotli + Postgres) — Implementation Plan

For agentic workers: REQUIRED SUB-SKILL: Use superpowers:subagent-driven-development (recommended) or superpowers:executing-plans to implement this plan task-by-task. Steps use checkbox (– []) syntax for tracking.

Goal: Build the new `lava-api-go/` Go service that ports all 13 `rutracker` proxy routes from the existing `Kotlin/Ktor :proxy` module to `Gin Gonic`, serving HTTP/3 with Brotli compression, backed by PostgreSQL (cache + audit + rate limit only), composing 14 vasic-digital submodules per the Decoupled Reusable Architecture rule.

Architecture: Single-binary Go service in one container. `submodules/http3` provides `quic-go` bridging; `submodules/cache/pkg/postgres` is the cache backend; `submodules/mdns` advertises `_lava-api._tcp` on port 8443; `submodules/observability` provides `slog` + `Prometheus` + `OTel Tempo`; `submodules/ratelimiter` and `submodules/recovery` provide sliding-window throttling and a `rutracker.org` circuit breaker; `submodules/security` provides PII redaction and HTTP security headers; `submodules/containers` orchestrates lifecycle; `submodules/auth` and `submodules/middleware` provide Gin glue. The 13 routes' wire shape is preserved byte-for-byte (Option A — behavioural parity); a hand-written OpenAPI 3.1 spec at `lava-api-go/api/openapi.yaml` is the source of truth, with `oapi-codegen` producing committed server interfaces and a typed client used by the cross-backend parity test.

Tech Stack: Go 1.24+; Gin Gonic; `quic-go v0.59` (via `digital.vasic.http3`); PostgreSQL 16 + `pgx/v5`; `golang-migrate`; `brotli`; `OpenTelemetry`; `Prometheus`; `Loki`; `Grafana`; `Tempo`; `oapi-codegen v2`; `podman/docker` (rootless); `k6` for load tests; `gosec`, `govulncheck`, `trivy` for security; `go-mutesting` for mutation testing.

Spec reference: Every architectural decision in this plan is locked in `docs/superpowers/specs/2026-04-28-sp2-go-api-migration-design.md` (commit 45fd9b9). When in doubt, read the spec — this plan is the *executable form* of that document.

Constitutional rules (already committed; non-negotiable):

- **Sixth Law** — every test provably falsifiable; rehearsal recorded in commit body.
- **Local-Only CI/CD** — no GitHub Actions, no GitLab pipelines; `lava-api-go/scripts/ci.sh` is the single gate.
- **Decoupled Reusable Architecture** — Lava-domain in `lava-api-go/internal/{auth,handlers,rutacker}` and `cmd/healthprobe`; everything else is thin glue around vasic-digital submodules.

Mirror policy: every commit on master of this repo **MUST** be pushed to **github + gitflic + gitlab + gitverse** before the next task starts.

Versioning: the Go API ships as `Lava-API-Go-2.0.0-2000` (semver 2.0.0, versionCode 2000). `scripts/tag.sh` already supports the existing android and api artifacts; this plan adds `api-go`.

Phases & commit boundaries

The plan has **14 phases**, each ending with a single logical commit pushed to all four upstreams.

Phase	Theme	Touces	Acceptance
0	Pre-flight: Ktor TXT records + plan kickoff	<code>proxy/...ServiceAdvertisement.kt</code> , <code>CLAUDE.md</code>	legacy proxy advertises symmetric TXT records, change pushed to all 4 upstreams
1	<code>lava-api-go/skeleton + go.mod + scripts/ci.sh</code> skeleton + docs	new files only	<code>cd lava-api-go && go vet ./...</code> passes on empty packages; <code>scripts/ci.sh --quick</code> runs to a clean exit spec validates; codegen produces server + client; <code>ci.sh</code> enforces "regenerate"
2	OpenAPI spec authoring + oapi-codegen wiring	<code>lava-api-go/api/</code> , <code>lava-api-go/internal/gen/</code>	

Phase	Theme	Touches	Acceptance
			produces empty diff"
3	Foundation: config + observability + server scaffold	lava-api-go/internal/ {config,observability,server}/	unit tests for config validation; server starts in tests; HTTP/3 self-test passes
4	Postgres migrations + cache + scripts	lava-api-go/migrations/, lava-api-go/ internal/cache/, scripts/migrate.sh, scripts/run-test-pg.sh	migrate up/ down clean; cache integration tests green against podman pg
5	Auth pass- through + rate limit + login throttle	lava-api-go/internal/ {auth,ratelimit}/	unit + integration tests; falsifiability rehearsal recorded
6	rutracker scraper (Lava-domain)	lava-api-go/internal/rutracker/	unit + fuzz tests; recorded fixtures replay deterministically
7	Route handlers (13 routes)	lava-api-go/internal/handlers/	one route at a time; each gets unit + contract test
8	mDNS advertisement glue + healthprobe binary	lava-api-go/internal/discovery/, lava-api-go/cmd/healthprobe/	localhost mDNS round-trip test passes
9	Main entry point + Dockerfile	lava-api-go/cmd/lava-api-go/main.go, lava-api-go/docker/Dockerfile	make image builds; container boots; /health returns 200 over HTTP/3
10	Test infrastructure: e2e + parity + load + contract	lava-api-go/tests/ {contract,e2e,parity,load,fixtures}/	parity test (Sixth Law load- bearing gate) green against

Phase	Theme	Touches	Acceptance
			running Ktor + Go containers
11	Container topology: profiles compose + observability + Lava CLI	docker-compose.yml, docker/observability/*, tools/lava-containers/...	./start.sh, --legacy, --both, --with-observability, --dev-docs all work
12	Lifecycle scripts: start/stop, build_and_release, tag.sh registry	start.sh, stop.sh, build_and_release.sh, scripts/tag.sh, docs/TAGGING.md	scripts/tag.sh --app api-go --dry-run produces Lava-API-Go-2.0.0-2000
13	Pretag verification + mutation testing + security	lava-api-go/scripts/{pretag-verify.sh,mutation.sh}, lava-api-go/SECURITY.md	scripted user flow recorded in .lava-ci-evidence/; mutation report archived
14	Acceptance + first tag + push	runs ./start.sh, executes 11 acceptance criteria, tags Lava-API-Go-2.0.0-2000	tag on all four upstreams; spec §18 checklist 100% green

Total tasks below: **63**. Each task has 3–8 bite-sized steps. Estimated total work: 5–8 engineer-days at full focus, longer if rutracker scraper edge cases bite.

Phase 0 — Pre-flight

This phase produces one commit before any new Go code lands. It updates the legacy Ktor proxy's mDNS advertisement so that once the Go API ships in Phase 14, the cross-backend parity test (and the Android client in SP-3) sees symmetric self-description on both backends.

Task 0.1: Add symmetric TXT records to ServiceAdvertisement.kt

Files:

- Modify: proxy/src/main/kotlin/digital/vasic/lava/api/discovery/ServiceAdvertisement.kt



Step 1: Read the existing file

```
cat proxy/src/main/kotlin/digital/vasic/lava/api/discovery/ServiceAdvertisement.kt
```

Note the existing JmDNS registration call — most likely `JmDNS.create(...).registerService(ServiceInfo.create(...))` with no TXT properties argument or an empty map.



Step 2: Modify the registration to publish TXT records

Per spec §8.3 the Ktor proxy must publish:

- engine=ktor
- version=<apiVersionName from proxy/build.gradle.kts>
- protocols=h11
- compression=identity
- tls=optional

Replace the existing `ServiceInfo.create(...)` call with one that includes a `Map<String, String>` of these key/values. Concretely, the JmDNS API:

```
val txt: Map<String, String> = mapOf(
    "engine" to "ktor",
    "version" to BuildConfig.VERSION,           // or read from
    gradle apiVersionName at build time
    "protocols" to "h11",
    "compression" to "identity",
    "tls" to "optional",
)
val info = ServiceInfo.create("_lava._tcp.local.", "Lava Proxy",
    port, 0, 0, txt)
jmdns.registerService(info)
```

If the version isn't already exposed via `BuildConfig`, hardcode "1.0.0" for now and file a follow-up TODO in `proxy/CLAUDE.md` to wire the build-time version. The version string is purely cosmetic for clients today — they read it but don't gate behavior on it.



Step 3: Build the proxy fat JAR to verify the change compiles

```
./gradlew :proxy:buildFatJar 2>&1 | tail -10
```

Expected: BUILD SUCCESSFUL. If unresolved imports: add import `javax.jmdns.ServiceInfo` (or whatever JmDNS package the existing code uses).



Step 4: Run the existing ServiceAdvertisementTest

```
./gradlew :proxy:test --tests  
    'digital.vasic.lava.api.discovery.ServiceAdvertisementTest'  
2>&1 | tail -10
```

Expected: PASS. If the test made assertions about the registered `ServiceInfo`, it may need to be extended to check that TXT records were set.



Step 5: Add a test asserting TXT records are present (Sixth Law)

In `proxy/src/test/kotlin/digital/vasic/lava/api/discovery/ServiceAdvertisementTest.kt`, add:

```
@Test  
fun `advertised service has symmetric TXT records`() {  
    val info = ServiceAdvertisement.buildServiceInfo(port = 8080)  
    assertEquals("ktor", info.getPropertyString("engine"))  
    assertEquals("h11", info.getPropertyString("protocols"))  
    assertEquals("identity",  
        info.getPropertyString("compression"))  
    assertEquals("optional", info.getPropertyString("tls"))  
    assertNotNull(info.getPropertyString("version"))  
}
```

(Assumes `ServiceAdvertisement` exposes a `buildServiceInfo()` factory — if it doesn't, refactor the existing `start(port)` to extract one.)



Step 6: Run new test, expect PASS

```
./gradlew :proxy:test --tests '*ServiceAdvertisementTest*' 2>&1 |  
tail
```



Step 7: Sixth Law falsifiability rehearsal

Temporarily mutate the production code: change "engine" to "ktor" to "engine" to "wrong". Re-run the test. Expected: FAIL with a clear message about engine. Revert. Run again, expect PASS.



Step 8: Commit

```
git add proxy/src/main/kotlin/digital/vasic/lava/api/discovery/
      ServiceAdvertisement.kt \
      proxy/src/test/kotlin/digital/vasic/lava/api/discovery/
      ServiceAdvertisementTest.kt
git commit -m "$(cat <<'EOF'
proxy: publish symmetric mDNS TXT records (engine=ktor, version,
protocols=h11, compression=identity, tls=optional) so
SP-2's cross-backend parity infrastructure can describe
both backends uniformly. New test asserts presence;
falsifiability rehearsal recorded – flipping engine=ktor
to engine=wrong fails the test with a clear engine-
mismatch assertion.
EOF
)"
```



Step 9: Push to all four upstreams

```
for r in github gitflic gitlab gitverse; do git push "$r" master;
done
```

Verify all four tips match.

Task 0.2: Cross-reference the SP-2 design doc from CLAUDE.md

Files:

- Modify: CLAUDE.md



Step 1: Append a "See also" pointer

Add to the existing "See also" section near the top of root CLAUDE.md:

```
> - `docs/superpowers/specs/2026-04-28-sp2-go-api-migration-
    design.md` – full SP-2 design doc (Go API service
    migration). Read this before touching `lava-api-go/`.
```



Step 2: Commit and push

```
git add CLAUDE.md
git commit -m
      "Cross-reference SP-2 design doc from root CLAUDE.md so
      future agents read the spec before touching lava-api-go/."
for r in github gitflic gitlab gitverse; do git push "$r" master;
done
```

Phase 1 — lava-api-go/ skeleton

Set up the directory tree, go.mod with all submodule replace directives, doc files inheriting constitutional rules, and a working (if empty) scripts/ci.sh. End state: the package compiles, go vet ./... passes, scripts/ci.sh --quick exits 0.

Task 1.1: Create directory tree and go.mod

Files:

- Create: lava-api-go/go.mod
- Create: lava-api-go/.gitignore



Step 1: Create the directory tree

```
mkdir -p lava-api-go/{api,cmd/lava-api-go,cmd/healthprobe,internal/{auth,cache,config,discovery,handlers,observability,ratelimit,rustracker,server,gen/client},migrations,tests/{contract,e2e,parity,load,fixtures/parity,observability/grafana/{provisioning/datasources,provisioning/dashboards,dashboards},tls,scripts}
```



Step 2: Write go.mod with replace directives for every consumed submodule

Create lava-api-go/go.mod:

```
module digital.vasic.lava.apigo
```

```
go 1.24
```

```
require (
    github.com/gin-gonic/gin v1.10.0
    github.com/jackc/pgx/v5 v5.9.2
    github.com/quic-go/quic-go v0.59.0
)
```

```
// vasic-digital submodules (frozen pins; replace from local mounts)
```

```
require (
    digital.vasic.auth v0.0.0
    digital.vasic.cache v0.0.0
    digital.vasic.challenges v0.0.0
    digital.vasic.config v0.0.0
    digital.vasic.containers v0.0.0
    digital.vasic.database v0.0.0
    digital.vasic.discovery v0.0.0
    digital.vasic.http3 v0.0.0
    digital.vasic.mdns v0.0.0
)
```



```

    digital.vasic.middleware v0.0.0
    digital.vasic.observability v0.0.0
    digital.vasic.ratelimiter v0.0.0
    digital.vasic.recovery v0.0.0
    digital.vasic.security v0.0.0
)

replace (
    digital.vasic.auth            => ../submodules/auth
    digital.vasic.cache           => ../submodules/cache
    digital.vasic.challenges      => ../submodules/challenges
    digital.vasic.config          => ../submodules/config
    digital.vasic.containers      => ../submodules/containers
    digital.vasic.database        => ../submodules/database
    digital.vasic.discovery       => ../submodules/discovery
    digital.vasic.http3           => ../submodules/http3
    digital.vasic.mdns            => ../submodules/mdns
    digital.vasic.middleware      => ../submodules/middleware
    digital.vasic.observability   => ../submodules/observability
    digital.vasic.ratelimiter     => ../submodules/ratelimiter
    digital.vasic.recovery        => ../submodules/recovery
    digital.vasic.security        => ../submodules/security
)

```



Step 3: Run `go mod tidy` and verify it resolves

```
cd lava-api-go && go mod tidy 2>&1 | tail -10
```

Expected: download messages, no errors. `go.sum` materialises.



Step 4: Write `.gitignore`

lava-api-go/.gitignore:

```

bin/
*.test
*.out
coverage.*
docker/tls/*.crt
docker/tls/*.key
.idea/
.vscode/
*.swp
.DS_Store
testdata/fuzz/.cache/

```



Step 5: Commit

```
git add lava-api-go/go.mod lava-api-go/go.sum lava-api-
go/.gitignore
git commit -m "lava-api-go: skeleton go.mod with replace
directives for all 14 consumed vasic-digital submodules
(frozen-pin via local mounts)."
for r in github gitflic gitlab gitverse; do git push "$r" master;
done
```

Task 1.2: Write the doc set (CONSTITUTION, README, CLAUDE, AGENTS, LICENSE, SECURITY)

Files:

- Create: lava-api-go/CONSTITUTION.md
- Create: lava-api-go/README.md
- Create: lava-api-go/CLAUDE.md
- Create: lava-api-go/AGENTS.md
- Create: lava-api-go/LICENSE
- Create: lava-api-go/SECURITY.md



Step 1: CONSTITUTION.md inherits SP-0 rules

lava-api-go/CONSTITUTION.md:

lava-api-go – Constitution

This module inherits the root project's constitutional rules. Modifying them in this module is forbidden; submodule constitutions may add stricter rules but MUST NOT relax these.

Inherited rules (non-negotiable)

- ****Sixth Law – Real User Verification**** – see root ``/CLAUDE.md``. Every test traverses the production code path, MUST be provably falsifiable (deliberate-mutation rehearsal recorded in PR body), MUST primary-assert on user-visible state, NOT on mock interaction counts. Cross-backend parity (test type 5) is the load-bearing release gate.
- ****Local-Only CI/CD**** – see root ``/CLAUDE.md``. No ``.github/workflows``, ``.gitlab-ci.yml``, etc. ``.scripts/ci.sh`` is the single local entry point.
- ****Decoupled Reusable Architecture**** – see root ``/CLAUDE.md``. Lava-domain only: ``.internal/auth``, ``.internal/handlers``, ``.internal/rutacker``, ``.cmd/healthprobe``. Everything else is thin glue around vasic-digital submodules.

Module-specific rules

- The module MUST refuse to start if Postgres is unreachable (P1 hard dependency, spec §7).

- The ``/metrics`` endpoint MUST be on a separate localhost-only listener; never on the public 8443 listener (spec §10.1).
- The ``Auth-Token`` header MUST be redacted from every log, span attribute, and audit field (the SHA-256 hash goes to ``request_audit.auth_realm_hash`` only).
- The cross-backend parity test (``tests/parity/``) is the load-bearing acceptance gate. A green parity suite means the Go API is byte-equivalent to the Ktor proxy on every supported request shape.
- Every release tag (``Lava-API-Go-X.Y.Z-NNNN``) requires a recorded entry in ``.lava-ci-evidence/`` from ``scripts/pretag-verify.sh`` against the exact commit being tagged.



Step 2: README.md (consumer-facing)

lava-api-go/README.md:

`# lava-api-go`

Go (Gin Gonic) implementation of the Lava API service. Replaces the legacy Kotlin/Ktor ``:proxy`` module's wire surface byte-for-byte while adding HTTP/3 (QUIC) transport, Brotli compression, PostgreSQL-backed response caching, request audit, and per-IP/per-route rate limiting.

The legacy Ktor proxy remains runnable as an opt-in fallback. The Android client (post-SP-3) speaks both APIs.

`## Quick start`

````bash`

`# Build`

`make build`

`# Run integration tests (requires podman or docker)`

`make test`

`# Full local CI gate`

`scripts/ci.sh`

`# Bring the service up via the orchestrator`

`cd ..`

`./start.sh`

`# default: api-go + postgres`

`./start.sh --with-observability`

`# + Prometheus + Loki +`

`Grafana + Tempo`

## Module path

digital.vasic.lava.apigo — distinct from the Kotlin proxy's digital.vasic.lava.api group.

## Wire contract

- HTTP/3 (QUIC) on UDP port 8443, with HTTP/2 fallback over TCP/8443.
- Brotli compression for clients sending Accept-Encoding: br.
- mDNS service-type `_lava-api._tcp` on port 8443 (vs the legacy `_lava._tcp` on 8080).
- TXT records: `engine=go, version=2.0.0, protocols=h3,h2, compression=br,gzip, tls=required, path=/.`
- 13 routes inherited from the legacy proxy (see `api/openapi.yaml`).

# Constitution

See CONSTITUTION.md and the root project's /CLAUDE.md. The Sixth Law is binding: every test must be provably falsifiable, every PR records a falsifiability rehearsal in its commit body, the cross-backend parity suite gates releases.

```
- [] **Step 3: CLAUDE.md (agent guide)**
```

```
`lava-api-go/CLAUDE.md` :
```

```
```markdown
```

```
# CLAUDE.md - lava-api-go
```

Agent-facing guide for working in this module.

```
## Inherited rules (non-negotiable)
```

See `CONSTITUTION.md`. Sixth Law, Local-Only CI/CD, Decoupled Reusable Architecture.

Tech stack

- Go 1.24+
- Gin Gonic, quic-go v0.59 (via `digital.vasic.http3`), pgx/v5
- Tests: stdlib `testing`, real Postgres in podman, real HTTP/3 client
- Local CI: `scripts/ci.sh` (no hosted CI ever)

Layout

- `api/openapi.yaml` - D1 spec-first, source of truth.
- `internal/gen/{server,client}/` - committed `oapi-codegen` output. CI enforces "regenerate produces empty diff".
- `internal/{auth,handlers,rutacker}/` and `cmd/healthprobe/` - Lava-domain code.
- `internal/{cache,config,discovery,observability,ratelimit,server}/` - thin glue around vasic-digital submodules.
- `migrations/` - golang-migrate SQL files; applied by the one-shot `lava-migrate` compose service.

- `tests/{contract,e2e,parity,load,fixtures}/` – the higher test types per spec §11.

When changing public API

The OpenAPI spec is the contract. Modify `api/openapi.yaml` first, regenerate, then update handlers. The cross-backend parity test will fail if the spec diverges from the Ktor proxy's wire shape – that's the protection.

Falsifiability protocol (Sixth Law clause 2)

Every PR adding or modifying a test MUST record in its commit body:

- `Test:` the test name + file
- `Mutation:` the deliberate production-code change
- `Observed:` the exact assertion message that fired
- `Reverted:` confirmation that the final commit reflects unmutated code



Step 4: AGENTS.md

lava-api-go/AGENTS.md:

AGENTS.md – lava-api-go

Local CI: `scripts/ci.sh`. Single source of truth.

Tools and gates

- `go vet ./...`, `go build ./...`, `go test -race -count=1 ./...`
- `oapi-codegen` invariant (regenerate; assert empty diff)
- `go mod tidy` invariant (sha256 before/after)
- Fuzz: `go test -fuzz=. -fuzztime=30s` per package with `Fuzz\*` tests
- gosec, govulncheck, trivy
- Quarterly: `scripts/mutation.sh` (go-mutesting)
- Pre-tag: `scripts/pretag-verify.sh` records `lava-ci-evidence/<commit>.json`

Workflow

1. Branch off `master` of the parent repo (Lava monorepo); the Go module lives at `lava-api-go/`.
2. Modify `api/openapi.yaml` first if changing wire shape.
3. Run `make generate` to update `internal/gen/`.
4. Implement the change.
5. `scripts/ci.sh` until green.
6.
For every test added/modified: run the falsifiability rehearsal (see `CLAUDE.md`). Record the result in the commit body.
7. Commit. Push to
`master` of the parent repo on github + gitflic + gitlab + gitverse.

Things to avoid

- Adding hosted-CI configuration (forbidden; constitutional).
- Importing Lava-Android-specific code (Android lives in ``app/``; Go API has no Android dep).
- Re-implementing functionality that exists in a vasic-digital submodule (Decoupled Reusable rule).
- Mocking internal Lava code in non-unit tests (Sixth Law clause 2).

Host Machine Stability Directive

Per ``/CLAUDE.md`` and propagated through ``submodules/*/CLAUDE.md``: never run commands that suspend, hibernate, sign-out, or kill the user session. Cap test parallelism (``GOMAXPROCS=2``, ``nice -n 19`` are recommended).



Step 5: LICENSE — same MIT as the rest of the project

Copy LICENSE from the repo root to lava-api-go/LICENSE.

```
cp LICENSE lava-api-go/LICENSE
```



Step 6: SECURITY.md placeholder

lava-api-go/SECURITY.md:

```
# Security findings tracker — lava-api-go
```

This file tracks ``gosec``, ``govulncheck``, and ``trivy`` findings that are known and accepted (with rationale). HIGH/CRITICAL findings hard-fail CI; lower severities or third-party-only vulnerabilities live here.

Severity	Tool	Source	Finding	Status	Rationale
(none yet)					

When adding an entry: include the tool's exact rule ID, the file/line, and a one-sentence explanation of why the finding is accepted. Re-evaluate quarterly (or when the rule's tool version changes).



Step 7: Commit

```
git add lava-api-go/CONSTITUTION.md lava-api-go/README.md lava-api-go/CLAUDE.md \
lava-api-go/AGENTS.md lava-api-go/LICENSE lava-api-go/SECURITY.md
```

```
git commit -m
    "lava-api-go: doc set (CONSTITUTION, README, CLAUDE,
    AGENTS, LICENSE, SECURITY) inheriting SP-0 constitutional
    rules; explicit per-module reiteration of the Sixth Law /
    Local-Only CI / Decoupled Reusable rules."
for r in github gitflic gitlab gitverse; do git push "$r" master;
done
```

Task 1.3: Initial Makefile + scripts/ci.sh skeleton

Files:

- Create: lava-api-go/Makefile
- Create: lava-api-go/scripts/ci.sh



Step 1: Makefile

lava-api-go/Makefile:

```
.PHONY: all build test ci generate migrate-up migrate-down image
        clean
```

```
all: build test
```

```
build:
```

```
    go build -o bin/lava-api-go ./cmd/lava-api-go
    go build -o bin/healthprobe ./cmd/healthprobe
```

```
test:
```

```
    go test -race -count=1 ./...
```

```
ci:
```

```
    ./scripts/ci.sh
```

```
generate:
```

```
    ./scripts/generate.sh
```

```
migrate-up:
```

```
    ./scripts/migrate.sh up
```

```
migrate-down:
```

```
    ./scripts/migrate.sh down
```

```
image:
```

```
    docker build -f docker/Dockerfile -t lava-api-go:dev .
```

```
clean:
```

```
    rm -rf bin/ coverage.* *.out
```



Step 2: scripts/ci.sh skeleton

lava-api-go/scripts/ci.sh:

```
#!/usr/bin/env bash
#
# scripts/ci.sh – single local entry point for lava-api-go CI.
# Per CONSTITUTION.md, this module does NOT use hosted CI
#   services.
#
# Usage:
#   scripts/ci.sh                # full gate
#   scripts/ci.sh --quick        # skip fuzz / gosec /
#                               govulncheck / load / image
#   scripts/ci.sh --fuzz-time=2m # override default 30s fuzz
#                               duration

set -Eeuo pipefail

ROOT="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"
cd "$ROOT"

QUICK=false
FUZZ_TIME="30s"
for arg in "$@"; do
  case "$arg" in
    --quick) QUICK=true ;;
    --fuzz-time=*) FUZZ_TIME="${arg#--fuzz-time=}" ;;
    -h|--help) sed -n '/^# Usage/,/^# */p' "$0" | sed 's/^#
    \?//'; exit 0 ;;
    *) echo "unknown flag: $arg" >&2; exit 2 ;;
  esac
done

log() { printf '\033[1;36m[ci]\033[0m %s\n' "$*"; }
fail() { printf '\033[1;31m[ci:fail]\033[0m %s\n' "$*" >&2; exit
1; }

# 1. go mod tidy invariant
log "step 1/N go mod tidy invariant"
_pre="$(sha256sum go.mod go.sum 2>/dev/null | sort)"
go mod tidy
_post="$(sha256sum go.mod go.sum 2>/dev/null | sort)"
[[ "$_pre" == "$_post" ]] || fail "go mod tidy produced a diff;
commit the tidied result"

# 2. go vet
log "step 2/N go vet ./..."
go vet ./...

# 3. go build
log "step 3/N go build ./..."
go build ./...

# Later phases append to this script as features land:
#   - oapi-codegen invariant
```



```
# - go test -race -count=1 ./...
# - fuzz
# - gosec
# - govulncheck
# - load (k6)
# - image build + trivy
#
# Steps are added in the phase that produces the corresponding
# code.

if $QUICK; then
    log "ci OK (quick - only steps 1-3)"
    exit 0
fi

log "ci OK"
```



Step 3: Make executable, run it

```
chmod +x lava-api-go/scripts/ci.sh
( cd lava-api-go && ./scripts/ci.sh --quick )
```

Expected: ci OK (quick - only steps 1-3). If go.mod/go.sum are missing, run `cd lava-api-go && go mod tidy` first.



Step 4: Commit

```
git add lava-api-go/Makefile lava-api-go/scripts/ci.sh
git commit -m "lava-api-go: initial Makefile and scripts/ci.sh
    skeleton (steps 1-3 wired: tidy invariant, vet, build).
    Later phases extend ci.sh with codegen invariant, tests,
    fuzz, gosec, govulncheck, load, image scan."
for r in github gitflic gitlab gitverse; do git push "$r" master;
done
```

Task 1.4: internal/version/version.go

Files:

- Create: lava-api-go/internal/version/version.go
- Create: lava-api-go/internal/version/version_test.go



Step 1: Write the file (this is the file `scripts/tag.sh` will read/write in Phase 12)

lava-api-go/internal/version/version.go:

```

// Package version exposes the build-time version constants for
// the lava-api-go
// service. scripts/tag.sh reads and rewrites these constants as
// part of the
// release tagging flow.
//
// Format invariants:
//   - Name MUST be a strict three-component semver:
//     MAJOR.MINOR.PATCH
//   - Code MUST be a positive integer that monotonically
//     increases per release
package version

const (
    // Name is the service's semver. Tag prefix: Lava-API-Go-
    // <Name>-<Code>.
    Name = "2.0.0"

    // Code is the integer release counter. New tags MUST
    // increment.
    Code = 2000
)

```



Step 2: Write a test asserting format invariants

lava-api-go/internal/version/version_test.go:

```

package version

import (
    "regexp"
    "testing"
)

var semverPat = regexp.MustCompile(`^\d+\.\d+\.\d+$`)

func TestNameIsStrictSemver(t *testing.T) {
    if !semverPat.MatchString(Name) {
        t.Fatalf("Name = %q, want strict MAJOR.MINOR.PATCH
            semver", Name)
    }
}

func TestCodeIsPositive(t *testing.T) {
    if Code <= 0 {
        t.Fatalf("Code = %d, want positive", Code)
    }
}

func TestNameAndCodeMatchInitialRelease(t *testing.T) {
    // Pinned to spec $15 – initial release is 2.0.0 / 2000.
    // When we cut a new release, this test gets bumped along with
    // the constants.
}

```

```

    if Name != "2.0.0" {
        t.Errorf("Name = %q, want 2.0.0 for initial SP-2
            release", Name)
    }
    if Code != 2000 {
        t.Errorf("Code = %d, want 2000 for initial SP-2 release",
            Code)
    }
}

```



Step 3: Run tests

```

( cd lava-api-go && go test -race -count=1 ./internal/
    version/... )

```

Expected: PASS.



Step 4: Sixth Law falsifiability rehearsal

Temporarily mutate Name = "2.0.0" to Name = "2.0". Re-run. Expected: TestNameIsStrictSemver fails. Revert. Re-run, expect PASS.



Step 5: Commit

```

git add lava-api-go/internal/version/
git commit -m "lava-api-go: internal/version package – Name/Code
    constants are the single source of truth for tag.sh;
    semver-format unit test verifies the invariants.
    Falsifiability rehearsal: mutating Name='2.0.0' to '2.0'
    fails TestNameIsStrictSemver as expected."
for r in github gitflic gitlab gitverse; do git push "$r" master;
done

```

Phase 2 — OpenAPI spec authoring + oapi-codegen

The OpenAPI spec is the wire contract. It's authored by hand by reading the existing Ktor route handlers and recording exactly what they emit. oapi-codegen produces server interfaces and a typed client; both are committed.

This is one of the highest-stakes phases — getting the spec wrong silently means the cross-backend parity test (Phase 10) fails for the wrong reason. Phase 7's handler implementations depend on this spec being correct.

Task 2.1: Author api/openapi.yaml from the Ktor route handlers

Files:

- Create: lava-api-go/api/openapi.yaml

This task is the largest single authoring step in the plan. Rather than reproducing the full ~600-line YAML inline, the procedure below tells you how to derive each route's entry from the corresponding Ktor source file. The full openapi.yaml committed at the end of this task MUST cover all 16 endpoints listed in the spec §6 routes table.



Step 1: Set up the file's preamble

lava-api-go/api/openapi.yaml:

openapi: 3.1.0

info:

title: Lava API

version: "2.0.0"

summary: Unofficial JSON API for the rutracker.org tracker.

description: |

Lava API is the JSON-REST surface consumed by the Lava Android client. It proxies and structures content from rutracker.org, applying response caching, audit logging, and per-IP rate limiting at the API tier.

This OpenAPI 3.1 document is the source of truth for the wire contract.

The legacy Kotlin/Ktor proxy (Lava-API 1.x, default port 8080, mDNS

`\_lava.\_tcp`) and the Go service (Lava-API-Go 2.x, default port 8443,

mDNS `\_lava-api.\_tcp`) MUST produce byte-equivalent responses for every

route below. This is enforced by `lava-api-go/tests/parity/`.

See `docs/superpowers/specs/2026-04-28-sp2-go-api-migration-design.md` for

the architectural backdrop and the constitutional rules that bind the implementation.

license:

name: MIT

identifier: MIT

servers:

- url: https://localhost:8443

```

    description: Local development (Go service over HTTP/3)
- url: http://localhost:8080
  description: Local development (legacy Kotlin proxy)

components:
  parameters:
    AuthToken:
      name: Auth-Token
      in: header
      schema:
        type: string
      description: Opaque rutracker session identifier (forwarded
        as cookie to upstream).
  schemas:
    Error:
      type: object
      required: [error]
      properties:
        error:
          type: string
        details:
          type: string
      # All concrete schemas (Forum, Topic, Comment, Torrent,
      # Favorite, etc.)
      # are defined below – populated as each route is authored.

paths:
  # populated below

```



Step 2: For each Ktor route file, derive its OpenAPI entry

Read each file in proxy/src/main/kotlin/digital/vasic/lava/api/routes/ and capture:

- HTTP method and path (exact pattern, including templating like /forum/{id})
- Required headers (almost all reads accept Auth-Token; writes require it)
- Path parameters and their types (id is integer)
- Query parameters and their types
- Form / JSON request bodies for POST routes
- Successful response status code (typically 200; /login may return 200 with cookie set; /comments/.../add may return 201)
- The @Serializable data class returned (these define response schemas)

For each route, append to paths: an entry like:

```

/forum:
  get:
    operationId: getForumTree
    summary: Top-level forum category tree.
    parameters:

```

```

- $ref: '#/components/parameters/AuthToken'
responses:
  '200':
    description: OK — paginated forum tree.
    headers:
      Content-Type:
        schema: { type: string, example: application/json }
    content:
      application/json:
        schema:
          $ref: '#/components/schemas/ForumTree'
  '401':
    description: Auth-Token missing or invalid.
    content:
      application/json:
        schema: { $ref: '#/components/schemas/Error' }

```

Repeat for every route in the table from spec §6:

```

GET /
GET /index
POST /login
GET /forum
GET /forum/{id}
GET /search
GET /topic/{id}
GET /topic2/{id}
GET /comments/{id}
POST /comments/{id}/add
GET /torrent/{id}
GET /download/{id}
GET /captcha/{path}
GET /favorites
POST /favorites/add/{id}
POST /favorites/remove/{id}

```

For each @Serializable Kotlin class referenced, define a matching components/schemas entry. The Kotlin file core/network/api/src/main/kotlin/lava/network/dto/ contains the existing DTOs — these are what the Android client expects on the wire today. Copy field names and types verbatim into the OpenAPI schemas. (Field rename or shape change here is a wire-breaking change; explicitly out of scope for SP-2.)



Step 3: Validate the spec with a YAML parser and a JSON-schema validator

```

cd lava-api-go
# Check YAML is valid
python3 -c "import yaml; yaml.safe_load(open('api/openapi.yaml'))" && echo "YAML OK"

```

```
# Optionally lint with `swagger-cli` or `redocly`. If neither is
  installed, skip.
# Most thorough: `redocly lint api/openapi.yaml` (npm install -g
  @redocly/cli)
```

Expected: YAML OK. If lint reports errors, fix them before moving on.



Step 4: Commit the spec

```
git add lava-api-go/api/openapi.yaml
git commit -m "lava-api-go: hand-author OpenAPI 3.1 spec from the
  Ktor proxy's route handlers and DTOs (16 endpoints, all
  schemas reverse-engineered from existing wire shapes). The
  cross-backend parity test (Phase 10) will fail loudly if
  the spec mis-encodes anything the Ktor proxy actually
  emits - that is the protection."
for r in github gitflic gitlab gitverse; do git push "$r" master;
done
```

Task 2.2: oapi-codegen config + scripts/generate.sh + initial generation

Files:

- Create: lava-api-go/api/codegen.yaml
- Create: lava-api-go/scripts/generate.sh
- Create: lava-api-go/internal/gen/server/api.gen.go (generated)
- Create: lava-api-go/internal/gen/client/api.gen.go (generated)



Step 1: oapi-codegen config

lava-api-go/api/codegen.yaml:

```
# Two output configs in one file (oapi-codegen v2 supports multi-
  config).
package: server
generate:
  models: true
  std-http-server: false
  gin-server: true
output: ../internal/gen/server/api.gen.go
output-options:
  skip-prune: false
  user-templates:
    # default templates; no overrides yet
---
package: client
generate:
  models: true
```

```
client: true
output: ../internal/gen/client/api.gen.go
```



Step 2: scripts/generate.sh

lava-api-go/scripts/generate.sh:

```
#!/usr/bin/env bash
set -Eeuo pipefail
ROOT="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"
cd "$ROOT"

# Pin oapi-codegen via go run -modfile so it's hermetic to this
# module's deps.
# Add github.com/oapi-codegen/oapi-codegen/v2 as a `tool`-only dep
# to go.mod.
go run github.com/oapi-codegen/oapi-codegen/v2/cmd/oapi-codegen \
    -config api/codegen.yaml api/openapi.yaml

go fmt ./internal/gen/...
echo "[generate] OK"

chmod +x lava-api-go/scripts/generate.sh
```



Step 3: Add oapi-codegen as a Go dep

```
cd lava-api-go
go get -tool github.com/oapi-codegen/oapi-codegen/v2/cmd/oapi-
    codegen@latest
go mod tidy
```



Step 4: Run generate

```
( cd lava-api-go && ./scripts/generate.sh )
```

Expected: [generate] OK and two non-empty files at internal/gen/server/api.gen.go and internal/gen/client/api.gen.go.



Step 5: Verify generated code compiles

```
( cd lava-api-go && go build ./internal/gen/... )
```

Expected: silent success.



Step 6: Add the codegen-invariant check to ci.sh

Insert into lava-api-go/scripts/ci.sh, after step 1 (the go mod tidy invariant):

```
# 2. oapi-codegen invariant
log "step 2/N oapi-codegen invariant"
_pre_gen="$(sha256sum internal/gen/server/api.gen.go internal/gen/
client/api.gen.go 2>/dev/null | sort)"
./scripts/generate.sh >/dev/null
_post_gen="$(sha256sum internal/gen/server/api.gen.go internal/
gen/client/api.gen.go 2>/dev/null | sort)"
[[ "$_pre_gen" == "$_post_gen" ]] || fail
    "oapi-codegen produced a diff; commit the regenerated
    files"
```

(Renumber subsequent steps as you go.)



Step 7: Run ci.sh --quick to confirm the invariant holds

```
( cd lava-api-go && ./scripts/ci.sh --quick )
```

Expected: clean exit.



Step 8: Commit

```
git add lava-api-go/api/codegen.yaml lava-api-go/scripts/
generate.sh \
lava-api-go/internal/gen/ lava-api-go/scripts/ci.sh \
lava-api-go/go.mod lava-api-go/go.sum
git commit -m "lava-api-go: oapi-codegen v2 wired (api/
codegen.yaml + scripts/generate.sh); generated server
interfaces and typed client committed at internal/gen/
{server,client}/. ci.sh enforces the codegen invariant via
sha256-before/after of the generated files."
for r in github gitflic gitlab gitverse; do git push "$r" master;
done
```

Phase 3 — Foundation: config + observability + server scaffold

These three packages are foundational for everything in Phase 4+. They have no Lava-domain logic but compose vasic-digital submodules into a unified application surface.

Task 3.1: internal/config — env-driven config with validation

Files:

- Create: lava-api-go/internal/config/config.go
- Create: lava-api-go/internal/config/config_test.go



Step 1: Write the test first (TDD)

lava-api-go/internal/config/config_test.go:

```
package config

import (
    "testing"
    "time"
)

func TestLoadFromEnvHappy(t *testing.T) {
    t.Setenv("LAVA_API_PG_URL", "postgres://
        lava:pwd@127.0.0.1:5432/lava_api?sslmode=disable")
    t.Setenv("LAVA_API_LISTEN", ":8443")
    t.Setenv("LAVA_API_METRICS_LISTEN", "127.0.0.1:9091")
    t.Setenv("LAVA_API_TLS_CERT", "/etc/lava-api-go/tls/
        server.crt")
    t.Setenv("LAVA_API_TLS_KEY", "/etc/lava-api-go/tls/
        server.key")
    t.Setenv("LAVA_API_OTLP_ENDPOINT", "http://127.0.0.1:4318")

    cfg, err := Load()
    if err != nil {
        t.Fatalf("Load: %v", err)
    }
    if cfg.PGUrl != "postgres://lava:pwd@127.0.0.1:5432/lava_api?
        sslmode=disable" {
        t.Errorf("PGUrl wrong: %q", cfg.PGUrl)
    }
    if cfg.Listen != ":8443" {
        t.Errorf("Listen wrong: %q", cfg.Listen)
    }
    if cfg.MetricsListen != "127.0.0.1:9091" {
        t.Errorf("MetricsListen wrong: %q", cfg.MetricsListen)
    }
    if cfg.AuditRetention != 30*24*time.Hour {
        t.Errorf("AuditRetention default wrong: %v",
            cfg.AuditRetention)
    }
}

func TestLoadRejectsMissingPGUrl(t *testing.T) {
```

```

t.Setenv("LAVA_API_PG_URL", "")
t.Setenv("LAVA_API_LISTEN", ":8443")
t.Setenv("LAVA_API_TLS_CERT", "/x.crt")
t.Setenv("LAVA_API_TLS_KEY", "/x.key")

_, err := Load()
if err == nil {
    t.Fatal("expected error when LAVA_API_PG_URL is empty")
}
}

func TestLoadRejectsMissingTLS(t *testing.T) {
    t.Setenv("LAVA_API_PG_URL", "postgres://x@y/z")
    t.Setenv("LAVA_API_LISTEN", ":8443")
    t.Setenv("LAVA_API_TLS_CERT", "")
    t.Setenv("LAVA_API_TLS_KEY", "")

    _, err := Load()
    if err == nil {
        t.Fatal("expected error when TLS cert/key paths are empty")
    }
}

```



Step 2: Run, expect FAIL (no Load yet)

```
( cd lava-api-go && go test ./internal/config/... )
```



Step 3: Implement config.go

lava-api-go/internal/config/config.go:

```

// Package config loads and validates the lava-api-go service
// configuration.
//
// Source of truth: environment variables (per Decoupled Reusable
// rule, this
// uses submodules/config under the hood – but the API surface
// here exposes
// only the fields lava-api-go actually consumes).
package config

import (
    "errors"
    "fmt"
    "os"
    "strconv"
    "time"
)

// Config is the validated runtime configuration.

```

```

type Config struct {
    // Postgres
    PGUrl    string
    PGSchema string

    // Listeners
    Listen        string // public LAN listener, e.g. ":8443"
    MetricsListen string // private metrics listener, e.g.
                        "127.0.0.1:9091"

    // TLS
    TLSCertPath string
    TLSKeyPath  string

    // Observability
    OTLPEndpoint string // optional; empty disables tracing export

    // Retention
    AuditRetention      time.Duration
    LoginRetention      time.Duration
    RateLimitOrphanTTL  time.Duration

    // mDNS
    MDNSInstanceName string
    MDNSServiceType   string
    MDNSPort           int

    // rutracker upstream
    RutrackerBaseURL string
}

// Load reads environment variables, applies defaults, and
// validates.
func Load() (*Config, error) {
    cfg := &Config{
        PGUrl:           os.Getenv("LAVA_API_PG_URL"),
        PGSchema:         envDefault("LAVA_API_PG_SCHEMA",
            "lava_api"),
        Listen:          envDefault("LAVA_API_LISTEN",
            ":8443"),
        MetricsListen:    envDefault("LAVA_API_METRICS_LISTEN", "127.0.0.1:9091"),
        TLSCertPath:      os.Getenv("LAVA_API_TLS_CERT"),
        TLSKeyPath:       os.Getenv("LAVA_API_TLS_KEY"),
        OTLPEndpoint:     os.Getenv("LAVA_API_OTLP_ENDPOINT"),
        AuditRetention:   envDuration("LAVA_API_AUDIT_RETENTION", 30*24*time.Hour),
        LoginRetention:   envDuration("LAVA_API_LOGIN_RETENTION", 7*24*time.Hour),
        RateLimitOrphanTTL: envDuration("LAVA_API_RATELIMIT_ORPHAN_TTL", 1*time.Hour),
        MDNSInstanceName: envDefault("LAVA_API_MDNS_INSTANCE",
            "Lava API"),
    }

```

```

        MDNSServiceType:    envDefault("LAVA_API_MDNS_TYPE",
            "_lava-api._tcp"),
        MDNSPort:           envInt("LAVA_API_MDNS_PORT", 8443),
        RutrackerBaseURL:   envDefault("LAVA_API_RUTRACKER_URL",
            "https://rutracker.org"),
    }

    if cfg.PGUrl == "" {
        return nil, errors.New("config: LAVA_API_PG_URL is
            required (P1 hard-dep)")
    }
    if cfg.TLSCertPath == "" || cfg.TLSKeyPath == "" {
        return nil, errors.New("config: LAVA_API_TLS_CERT and
            LAVA_API_TLS_KEY are required")
    }
    if cfg.Listen == "" {
        return nil, errors.New("config: LAVA_API_LISTEN is
            required")
    }
    if cfg.MDNSPort <= 0 || cfg.MDNSPort > 65535 {
        return nil, fmt.Errorf("config: LAVA_API_MDNS_PORT %d out
            of range", cfg.MDNSPort)
    }
    return cfg, nil
}

func envDefault(key, def string) string {
    if v := os.Getenv(key); v != "" {
        return v
    }
    return def
}

func envInt(key string, def int) int {
    if v := os.Getenv(key); v != "" {
        if n, err := strconv.Atoi(v); err == nil {
            return n
        }
    }
    return def
}

func envDuration(key string, def time.Duration) time.Duration {
    if v := os.Getenv(key); v != "" {
        if d, err := time.ParseDuration(v); err == nil {
            return d
        }
    }
    return def
}

```



Step 4: Run tests, expect PASS

```
( cd lava-api-go && go test -race -count=1 ./internal/config/... )
```



Step 5: Falsifiability rehearsal

Mutate `if cfg.PGUrl == "" { return nil, errors.New(...) }` to `if false { ... }`. Re-run `TestLoadRejectsMissingPGUrl`. Expected: FAIL. Revert.



Step 6: Commit

```
git add lava-api-go/internal/config/
git commit -m "lava-api-go: internal/config - env-driven Config
    with validation. P1 hard-dep enforced (LAVA_API_PG_URL
    required), TLS cert/key paths required, MDNSPort range
    checked. Falsifiability: mutating the PGUrl-required check
    to no-op fails TestLoadRejectsMissingPGUrl."
for r in github gitflic gitlab gitverse; do git push "$r" master;
done
```

Task 3.2: internal/observability — logger + metrics + tracing + health

Files:

- Create: `lava-api-go/internal/observability/log.go`
- Create: `lava-api-go/internal/observability/metrics.go`
- Create: `lava-api-go/internal/observability/tracing.go`
- Create: `lava-api-go/internal/observability/health.go`
- Create: `lava-api-go/internal/observability/observability_test.go`

This task is content-heavy because it wires four submodules. Use the upstream READMEs (`submodules/observability/README.md`, `submodules/security/pkg/pii/`) as the API reference.



Step 1: log.go — slog-style logger with PII redaction

`lava-api-go/internal/observability/log.go`:

```
// Package observability glues the vasic-digital observability +
    security
// modules into a single setup callable from cmd/lava-api-go.
package observability

import (
    "io"
    "log/slog"
    "os"
```

```

    obslog "digital.vasic.observability/pkg/logging"
    "digital.vasic.security/pkg/pii"
)

// LogConfig configures the structured logger.
type LogConfig struct {
    Output      io.Writer // default os.Stdout
    Level       slog.Level
    RedactKeys []string  // attribute names whose values are
                redacted before write
}

// DefaultRedactKeys is the redaction denylist applied across slog
// attributes and OTel span attributes for lava-api-go. Per spec
// §9.
var DefaultRedactKeys = []string{
    "Auth-Token", "auth_token",
    "Cookie", "cookie",
    "Set-Cookie", "set_cookie",
    "Authorization", "authorization",
    "X-Auth", "x_auth",
}

// NewLogger returns a slog.Logger that writes JSON-formatted
// records,
// with values for any attribute name in RedactKeys replaced by
// "[REDACTED]".
func NewLogger(cfg LogConfig) *slog.Logger {
    if cfg.Output == nil {
        cfg.Output = os.Stdout
    }
    if len(cfg.RedactKeys) == 0 {
        cfg.RedactKeys = DefaultRedactKeys
    }
    redactor := pii.NewKeyRedactor(cfg.RedactKeys)
    return obslog.NewJSON(cfg.Output, cfg.Level,
        obslog.WithRedactor(redactor))
}

```

Note for the implementer: the exact API of `digital.vasic.observability/pkg/logging` and `digital.vasic.security/pkg/pii` may differ from what's shown above. Read the actual upstream sources at `submodules/observability/pkg/logging/` and `submodules/security/pkg/pii/` and adapt accordingly. The contract this file MUST satisfy is "returns a `slog.Logger` that writes JSON-formatted records and redacts the keys in `DefaultRedactKeys`".



Step 2: metrics.go — Prometheus collectors

`lava-api-go/internal/observability/metrics.go`:

```

package observability

import (
    "net/http"

    obsmetrics "digital.vasic.observability/pkg/metrics"
    "github.com/gin-gonic/gin"
)

// Metrics is the lava-api-go-specific metrics collector set.
//
// The named collectors below are required by the spec (§10.1) –
// the
// names are part of the operator-visible contract. Renaming them
// is a
// breaking change for dashboards and alert rules.
type Metrics struct {
    HTTPRequestsTotal          obsmetrics.Counter
    HTTPRequestDurationSeconds obsmetrics.Histogram
    CacheOutcomeTotal          obsmetrics.Counter
    RateLimitBlockedTotal      obsmetrics.Counter
    RutrackerUpstreamDuration  obsmetrics.Histogram
    RutrackerUpstreamErrors    obsmetrics.Counter
    DBPoolAcquireDuration      obsmetrics.Histogram
    MDNSAdvertisementActive     obsmetrics.Gauge
}

// NewMetrics registers all collectors with the default Prometheus
// registry.
func NewMetrics() *Metrics {
    r := obsmetrics.DefaultRegistry()
    return &Metrics{
        HTTPRequestsTotal:
            r.Counter("http_requests_total", "Count of HTTP requests
            by method, route, status.", []string{"method", "route",
            "status"}),
        HTTPRequestDurationSeconds:
            r.Histogram("http_request_duration_seconds",
            "HTTP request handling latency.", []string{"method",
            "route"}, obsmetrics.DefaultBuckets),
        CacheOutcomeTotal:
            r.Counter("cache_outcome_total", "Cache outcome per
            request: hit | miss | bypass | invalidate.",
            []string{"outcome"}),
        RateLimitBlockedTotal:
            r.Counter("rate_limit_blocked_total",
            "Requests blocked by rate limiter, by route_class.",
            []string{"route_class"}),
        RutrackerUpstreamDuration:
            r.Histogram("rutracker_upstream_duration_seconds",
            "Latency of upstream calls to rutracker.org by route.",
            []string{"route"}, obsmetrics.DefaultBuckets),
    }
}

```



```

    RutrackerUpstreamErrors:
        r.Counter("rutracker_upstream_errors_total", "Upstream
errors hitting rutracker.org by route and kind.",
[]string{"route", "kind"}),
    DBPoolAcquireDuration:
        r.Histogram("db_pool_acquire_duration_seconds", "Time to
acquire a Postgres connection from the pool.", nil,
obsmetrics.DefaultBuckets),
    MDNSAdvertisementActive:
        r.Gauge("mdns_advertisement_active", "1 if the mDNS
advertisement is active, 0 otherwise.", nil),
    }
}

// MetricsHandler returns the http.Handler exposing /metrics.
// Mount on the
// dedicated localhost listener – never on the public API
// listener.
func MetricsHandler() http.Handler {
    return obsmetrics.HTTPHandler()
}

// GinMiddleware records request count + duration per route.
func (m *Metrics) GinMiddleware() gin.HandlerFunc {
    return
        obsmetrics.GinMiddleware(obsmetrics.GinMiddlewareConfig{
            Counter:    m.HTTPRequestsTotal,
            Histogram:  m.HTTPRequestDurationSeconds,
        })
}

```

Note: as with log.go, adapt to the actual upstream API at
submodules/observability/pkg/{metrics,gin}/.



Step 3: tracing.go — OTel exporter

lava-api-go/internal/observability/tracing.go:

```

package observability

import (
    "context"
    "fmt"

    obstrace "digital.vasic.observability/pkg/trace"
)

// TracerConfig configures the OTel tracer.
type TracerConfig struct {
    ServiceName      string
    ServiceVersion    string
    OTLPEndpoint      string // empty = no exporter (logging-only
tracer)
}

```

```

    SampleRate    float64
}

// Tracer is the active tracer; close via Shutdown to flush
// pending spans.
type Tracer struct {
    t obstrace.Tracer
}

func NewTracer(ctx context.Context, cfg TracerConfig) (*Tracer,
    error) {
    if cfg.SampleRate <= 0 {
        cfg.SampleRate = 1.0
    }
    t, err := obstrace.InitTracer(&obstrace.TracerConfig{
        ServiceName:    cfg.ServiceName,
        ServiceVersion: cfg.ServiceVersion,
        ExporterType:    exporterFor(cfg.OTLPEndpoint),
        ExporterEndpoint: cfg.OTLPEndpoint,
        SampleRate:      cfg.SampleRate,
    })
    if err != nil {
        return nil, fmt.Errorf("observability: tracer init: %w",
            err)
    }
    return &Tracer{t: t}, nil
}

func (tr *Tracer) Shutdown(ctx context.Context) error {
    return tr.t.Shutdown(ctx)
}

func exporterFor(endpoint string) obstrace.ExporterType {
    if endpoint == "" {
        return obstrace.ExporterStdout
    }
    return obstrace.ExporterOTLPHTTP
}

```



Step 4: health.go — /health and /ready handlers

lava-api-go/internal/observability/health.go:

```

package observability

import (
    "context"
    "net/http"

    "github.com/gin-gonic/gin"
)

```

```

// ReadinessProbe is invoked by /ready. It returns nil when the
// service
// can serve traffic, or an error describing why not. Concrete
// probes
// are wired in cmd/lava-api-go/main.go (DB ping, breaker state).
type ReadinessProbe func(ctx context.Context) error

// LivenessHandler always returns 200 if the process is up.
func LivenessHandler() gin.HandlerFunc {
    return func(c *gin.Context) {
        c.JSON(http.StatusOK, gin.H{"status": "alive"})
    }
}

// ReadinessHandler returns 200 if probe(ctx) returns nil; 503
// otherwise.
func ReadinessHandler(probe ReadinessProbe) gin.HandlerFunc {
    return func(c *gin.Context) {
        if err := probe(c.Request.Context()); err != nil {
            c.JSON(http.StatusServiceUnavailable, gin.H{
                "status": "not ready",
                "error": err.Error(),
            })
            return
        }
        c.JSON(http.StatusOK, gin.H{"status": "ready"})
    }
}

```



Step 5: observability_test.go – unit tests

Write tests for:

- NewLogger redacts every key in DefaultRedactKeys (write a record with Auth-Token: secret, capture stdout, assert secret does not appear; assert [REDACTED] does).
- LivenessHandler returns 200.
- ReadinessHandler returns 200 for a probe returning nil and 503 for a probe returning an error.

Use `httptest.NewRecorder()` and a Gin test engine to drive the handlers.



Step 6: Run tests, expect PASS

```
( cd lava-api-go && go test -race -count=1 ./internal/
  observability/... )
```



Step 7: Falsifiability rehearsal – log redaction

In log.go, mutate `cfg.RedactKeys = DefaultRedactKeys` to `cfg.RedactKeys = nil`. Re-run the redaction test. Expected: FAIL because secret now appears in the captured log line. Revert.



Step 8: Commit

```
git add lava-api-go/internal/observability/
git commit -m "lava-api-go: internal/observability - slog logger
with PII-key redaction (Auth-Token / Cookie /
Authorization), Prometheus metric set per spec §10.1, OTel
tracer wiring with optional OTLP HTTP exporter, /health
(liveness, no DB) and /ready (probe-driven) Gin handlers.
Falsifiability: nil'ing the redactor key list causes the
auth-token to leak into log output and fails the redaction
unit test."
for r in github gitflic gitlab gitverse; do git push "$r" master;
done
```

Task 3.3: internal/server — Gin engine + HTTP/3 wiring + dual listener

Files:

- Create: lava-api-go/internal/server/server.go
- Create: lava-api-go/internal/server/server_test.go



Step 1: Write the test first

lava-api-go/internal/server/server_test.go:

```
package server

import (
    "context"
    "crypto/tls"
    "crypto/x509"
    "io"
    "net/http"
    "strings"
    "testing"
    "time"

    "github.com/gin-gonic/gin"
    "github.com/quic-go/quic-go/http3"
)

func TestServerStartAndServeHTTP3(t *testing.T) {
    gin.SetMode(gin.TestMode)
    r := gin.New()
```

```

r.GET("/_self_test", func(c *gin.Context) {
    c.String(http.StatusOK, "ok") })

tlsConf, certBytes := selfSignedTLS(t)

srv, err := New(Config{
    Listen:      "127.0.0.1:0",
    MetricsListen: "127.0.0.1:0",
    Engine:      r,
    MetricsHandler: http.HandlerFunc(func(w
        http.ResponseWriter, _ *http.Request) {
            w.Write([]byte("# metrics ok"))
        }),
    TLSConfig: tlsConf,
})
if err != nil { t.Fatalf("New: %v", err) }

go func() { _ = srv.Start() }()
t.Cleanup(func() {
    ctx, cancel := context.WithTimeout(context.Background(),
        2*time.Second)
    defer cancel()
    _ = srv.Shutdown(ctx)
})
time.Sleep(300 * time.Millisecond) // wait for bind

cli := &http.Client{Transport: &http3.Transport{
    TLSClientConfig: clientTLS(certBytes),
}, Timeout: 3*time.Second}
resp, err := cli.Get("https://" + srv.Addr() + "/_self_test")
if err != nil { t.Fatalf("h3 get: %v", err) }
defer resp.Body.Close()
body, _ := io.ReadAll(resp.Body)
if string(body) != "ok" {
    t.Fatalf("body=%q want ok", string(body))
}
if !strings.HasPrefix(resp.Proto, "HTTP/3") {
    t.Errorf("proto=%q want HTTP/3.x", resp.Proto)
}
}

// helpers selfSignedTLS / clientTLS adapted from submodules/
// http3/internal/testcert/.
func selfSignedTLS(t *testing.T) (*tls.Config, []byte) {
    // ... (use the same approach as submodules/http3/internal/
    // testcert/testcert.go;
    // paste-in copy is acceptable in test code per the
    // existing test convention)
}

func clientTLS(certBytes []byte) *tls.Config {
    pool := x509.NewCertPool()
    pool.AppendCertsFromPEM(certBytes)

```

```

    return &tls.Config{RootCAs: pool, NextProtos: []string{"h3"}}
}

```



Step 2: Run, expect FAIL (no Server type)

```
( cd lava-api-go && go test ./internal/server/... )
```



Step 3: Implement server.go using submodules/http3

lava-api-go/internal/server/server.go:

```

// Package server hosts a Gin engine over HTTP/3 (via submodules/
// http3) on the
// public listener and a separate plain-HTTP /metrics listener on
// localhost.

```

```
package server
```

```

import (
    "context"
    "crypto/tls"
    "errors"
    "net"
    "net/http"
    "sync"

    "github.com/gin-gonic/gin"

    h3 "digital.vasic.http3/pkg/server"
)

```

```
// Config configures a Server.
```

```

type Config struct {
    Listen      string          // public listener, e.g. ":8443"
    MetricsListen
        string          // private metrics listener, e.g.
        "127.0.0.1:9091"
    Engine      *gin.Engine
    MetricsHandler http.Handler
    TLSConfig    *tls.Config
}

```

```
// Server hosts both listeners.
```

```

type Server struct {
    cfg      Config
    h3srv     *h3.Server
    metrics   *http.Server
    mu        sync.Mutex
    stopped   bool
}

```

```
func New(cfg Config) (*Server, error) {
```

```

if cfg.Engine == nil {
    return nil, errors.New("server: Engine is required")
}
if cfg.MetricsHandler == nil {
    return nil, errors.New("server: MetricsHandler is
    required")
}
if cfg.TLSConfig == nil {
    return nil, errors.New("server: TLSConfig is required")
}
h3srv, err := h3.New(h3.Config{
    Addr:    cfg.Listen,
    Handler: cfg.Engine,
    TLSConf: cfg.TLSConfig,
})
if err != nil { return nil, err }
return &Server{
    cfg:    cfg,
    h3srv:  h3srv,
    metrics: &http.Server{Addr: cfg.MetricsListen, Handler:
    cfg.MetricsHandler},
}, nil
}

```

// Start binds both listeners and serves until Shutdown.

```

func (s *Server) Start() error {
    errCh := make(chan error, 2)
    go func() { errCh <- s.h3srv.Start() }()
    go func() {
        ln, err := net.Listen("tcp", s.cfg.MetricsListen)
        if err != nil { errCh <- err; return }
        errCh <- s.metrics.Serve(ln)
    }()
    return <-errCh
}

```

```

func (s *Server) Shutdown(ctx context.Context) error {
    s.mu.Lock()
    if s.stopped { s.mu.Unlock(); return nil }
    s.stopped = true
    s.mu.Unlock()
    _ = s.metrics.Shutdown(ctx)
    return s.h3srv.Shutdown(ctx)
}

```

```

func (s *Server) Addr() string { return s.cfg.Listen }

```



Step 4: Run tests, expect PASS

```
( cd lava-api-go && go test -race -count=1 ./internal/server/... )
```



Step 5: Falsifiability rehearsal

Mutate the route registration line in the test from `r.GET("/_self_test", ...)` to `r.GET("/wrong", ...)`. Re-run; expected FAIL with status mismatch (404 vs ok). Revert.



Step 6: Add `go test -race -count=1 ./...` to `ci.sh`

In `lava-api-go/scripts/ci.sh` add (after the build step, before the `--quick exit`):

```
log "step N/N go test -race -count=1 ./..."
go test -race -count=1 ./...
```

(Renumber the steps to match the actual count.)



Step 7: Commit

```
git add lava-api-go/internal/server/ lava-api-go/scripts/ci.sh
git commit -m "lava-api-go: internal/server - Gin engine fronted
    by submodules/http3 on public listener (:8443 default)
    plus dedicated localhost-only metrics listener (:9091
    default). Round-trip test exercises real HTTP/3 client →
    server. Falsifiability: route-pattern mutation produces
    404 vs expected response. ci.sh now runs the full test
    suite (test step appended)."
for r in github gitflic gitlab gitverse; do git push "$r" master;
done
```

Phase 4 — Postgres migrations + cache wrapper + scripts

Task 4.1: golang-migrate SQL migrations

Files:

- Create: `lava-api-go/migrations/0001_response_cache.{up,down}.sql`
- Create: `lava-api-go/migrations/0002_request_audit.{up,down}.sql`
- Create: `lava-api-go/migrations/0003_rate_limit_bucket.{up,down}.sql`
- Create: `lava-api-go/migrations/0004_login_attempt.{up,down}.sql`



Step 1: `0001_response_cache.up.sql` (full DDL from spec §7)

-- 0001_response_cache.up.sql

```
CREATE SCHEMA IF NOT EXISTS lava_api;
```

```
CREATE TABLE IF NOT EXISTS lava_api.response_cache (  
    cache_key      TEXT PRIMARY KEY,  
    upstream_status SMALLINT NOT NULL,  
    body_brotli    BYTEA NOT NULL,  
    content_type   TEXT NOT NULL,  
    fetched_at     TIMESTAMPTZ NOT NULL DEFAULT now(),  
    expires_at     TIMESTAMPTZ NOT NULL,  
    hit_count      BIGINT NOT NULL DEFAULT 0  
);
```

```
CREATE INDEX IF NOT EXISTS response_cache_expires_at_idx  
ON lava_api.response_cache (expires_at);
```



Step 2: 0001_response_cache.down.sql

```
DROP INDEX IF EXISTS lava_api.response_cache_expires_at_idx;  
DROP TABLE IF EXISTS lava_api.response_cache;
```



Step 3: 0002_request_audit.up.sql

```
CREATE TABLE IF NOT EXISTS lava_api.request_audit (  
    id          BIGSERIAL PRIMARY KEY,  
    received_at TIMESTAMPTZ NOT NULL DEFAULT now(),  
    method      TEXT NOT NULL,  
    path        TEXT NOT NULL,  
    query       TEXT,  
    client_ip   INET,  
    auth_realm_hash TEXT,  
    upstream_status SMALLINT,  
    upstream_ms  INTEGER,  
    cache_outcome TEXT NOT NULL,  
    bytes_out    INTEGER  
);
```

```
CREATE INDEX IF NOT EXISTS request_audit_received_at_idx  
ON lava_api.request_audit (received_at);
```



Step 4: 0002_request_audit.down.sql

```
DROP INDEX IF EXISTS lava_api.request_audit_received_at_idx;  
DROP TABLE IF EXISTS lava_api.request_audit;
```



Step 5: 0003_rate_limit_bucket.up.sql

```
CREATE TABLE IF NOT EXISTS lava_api.rate_limit_bucket (
  client_ip      INET NOT NULL,
  route_class    TEXT NOT NULL,
  tokens         DOUBLE PRECISION NOT NULL,
  last_refill_at TIMESTAMPTZ NOT NULL,
  PRIMARY KEY (client_ip, route_class)
);
```



Step 6: 0003_rate_limit_bucket.down.sql

```
DROP TABLE IF EXISTS lava_api.rate_limit_bucket;
```



Step 7: 0004_login_attempt.up.sql

```
CREATE TABLE IF NOT EXISTS lava_api.login_attempt (
  id          BIGSERIAL PRIMARY KEY,
  client_ip   INET NOT NULL,
  username_hash TEXT NOT NULL,
  succeeded    BOOLEAN NOT NULL,
  attempted_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

```
CREATE INDEX IF NOT EXISTS login_attempt_lookup_idx
  ON lava_api.login_attempt (client_ip, attempted_at DESC);
```



Step 8: 0004_login_attempt.down.sql

```
DROP INDEX IF EXISTS lava_api.login_attempt_lookup_idx;
DROP TABLE IF EXISTS lava_api.login_attempt;
```



Step 9: Commit (no test yet — test happens in Task 4.3 against a live DB)

```
git add lava-api-go/migrations/
git commit -m "lava-api-go: golang-migrate SQL migrations for the
  four runtime tables (response_cache, request_audit,
  rate_limit_bucket, login_attempt). Schema lava_api created
  idempotently. Down migrations clean up tables and
  indexes."
for r in github gitflic gitlab gitverse; do git push "$r" master;
done
```

Task 4.2: scripts/migrate.sh and scripts/run-test-pg.sh

Files:

- Create: lava-api-go/scripts/migrate.sh

- Create: lava-api-go/scripts/run-test-pg.sh



Step 1: migrate.sh

lava-api-go/scripts/migrate.sh:

```
#!/usr/bin/env bash
#
# scripts/migrate.sh – apply or roll back golang-migrate
# migrations against
# the Postgres database identified by LAVA_API_PG_URL.
#
# Usage:
#   scripts/migrate.sh up           # apply all pending
#   scripts/migrate.sh down 1       # roll back N steps
#   scripts/migrate.sh version      # print current version
#   scripts/migrate.sh force VERSION # force-set version after
#                                   manual repair

set -Eeuo pipefail
ROOT="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"
cd "$ROOT"

: "${LAVA_API_PG_URL:?must be set, e.g. postgres://
    lava:pwd@127.0.0.1:5432/lava_api?sslmode=disable}"

go run -tags 'postgres' github.com/golang-migrate/migrate/v4/cmd/
    migrate \
    -path migrations \
    -database "$LAVA_API_PG_URL" \
    "$@"

chmod +x lava-api-go/scripts/migrate.sh
```



Step 2: run-test-pg.sh — adapt the same script as submodules/ cache/scripts/run-postgres-test.sh

Read submodules/cache/scripts/run-postgres-test.sh (committed in Step 5b of this session). Copy verbatim with two changes:

1. Replace test-runner command with `go test -race -count=1 -v ./internal/cache/... ./tests/integration/...`
2. Bump container name prefix to `lava-api-go-pg-test-$$`.



Step 3: Add migrate dep to go.mod

```
cd lava-api-go
go get -tool github.com/golang-migrate/migrate/v4/cmd/
    migrate@latest
go mod tidy
```



Step 4: Smoke test — run scripts/run-test-pg.sh

```
( cd lava-api-go && ./scripts/run-test-pg.sh )
```

Even with no tests yet, the script must launch a podman Postgres, set POSTGRES_TEST_URL, and exit 0 (or skip cleanly because no tests match the pattern). Adjust the script if it fails to start podman.



Step 5: Apply migrations against the test database to verify they're valid

```
( cd lava-api-go && \
  CONTAINER_NAME="lava-test-$(date +%s)" && \
  podman run --rm -d --name "$CONTAINER_NAME" -p 65432:5432 \
    -e POSTGRES_PASSWORD=pwd -e POSTGRES_USER=lava -e \
      POSTGRES_DB=lava_api \
    postgres:16-alpine && \
  sleep 5 && \
  LAVA_API_PG_URL="postgres://lava:pwd@127.0.0.1:65432/lava_api? \
    sslmode=disable" \
    ./scripts/migrate.sh up && \
  LAVA_API_PG_URL="postgres://lava:pwd@127.0.0.1:65432/lava_api? \
    sslmode=disable" \
    ./scripts/migrate.sh down 4 && \
  podman rm -f "$CONTAINER_NAME" \
)
```

Expected: migrations apply cleanly, then roll back cleanly.



Step 6: Commit

```
git add lava-api-go/scripts/migrate.sh lava-api-go/scripts/run-
    test-pg.sh \
    lava-api-go/go.mod lava-api-go/go.sum
git commit -m "lava-api-go: scripts/migrate.sh (golang-migrate
    wrapper) and scripts/run-test-pg.sh (transient podman
    Postgres for integration tests, modeled on submodules/
    cache/scripts/run-postgres-test.sh). Migrations verified
    up + down 4 cleanly against a live Postgres 16 container."
for r in github gitflic gitlab gitverse; do git push "$r" master;
done
```

Task 4.3: internal/cache — wrapper around submodules/cache/pkg/postgres

Files:

- Create: lava-api-go/internal/cache/cache.go
- Create: lava-api-go/internal/cache/cache_test.go
- Create: lava-api-go/internal/cache/integration_test.go



Step 1: Implement cache.go as a thin wrapper

lava-api-go/internal/cache/cache.go:

```
// Package cache adapts submodules/cache/pkg/postgres for lava-
// api-go's
// concrete needs: cache key construction (per spec §6) and
// outcome
// classification (hit | miss | bypass | invalidate) for metrics.
package cache

import (
    "context"
    "crypto/sha256"
    "encoding/hex"
    "sort"
    "strings"
    "time"

    pgcache "digital.vasic.cache/pkg/postgres"
)

// Key constructs the canonical cache key from a request's
// identity,
// per spec §6's normalisation rule.
func Key(method, routeTemplate string, pathVars
    map[string]string, query map[string][]string,
    authRealmHash string) string {
    pathKeys := make([]string, 0, len(pathVars))
    for k := range pathVars {
        pathKeys = append(pathKeys, k)
    }
    sort.Strings(pathKeys)
    var pathParts []string
    for _, k := range pathKeys {
        pathParts = append(pathParts, k+"="+pathVars[k])
    }
    queryKeys := make([]string, 0, len(query))
    for k := range query {
        queryKeys = append(queryKeys, strings.ToLower(k))
    }
    sort.Strings(queryKeys)
```

```

var queryParts []string
for _, k := range queryKeys {
    vs := query[k]
    if len(vs) == 0 {
        continue
    }
    // Sort values for stability where order doesn't matter
    // (the wire shape
    // dictates this; revisit if a route is order-sensitive).
    sortedVs := append([]string(nil), vs...)
    sort.Strings(sortedVs)
    queryParts = append(queryParts,
        k+"="+strings.Join(sortedVs, ","))
}
if authRealmHash == "" {
    authRealmHash = "anon"
}
raw := strings.Join([]string{
    method,
    routeTemplate,
    strings.Join(pathParts, ";"),
    strings.Join(queryParts, "&"),
    authRealmHash,
}, "\n")
sum := sha256.Sum256([]byte(raw))
return hex.EncodeToString(sum[:])
}

// Outcome classifies a cache lookup result.
type Outcome string

const (
    OutcomeHit      Outcome = "hit"
    OutcomeMiss     Outcome = "miss"
    OutcomeBypass   Outcome = "bypass"
    OutcomeInvalidate Outcome = "invalidate"
)

// Client is the lava-api-go cache facade. It owns a submodule/
// cache pgcache.Client
// and exposes only the operations handlers need.
type Client struct {
    inner *pgcache.Client
}

func New(inner *pgcache.Client) *Client { return &Client{inner:
    inner} }

func (c *Client) Get(ctx context.Context, key string) ([]byte,
    Outcome, error) {
    v, err := c.inner.Get(ctx, key)
    if err != nil {
        return nil, OutcomeBypass, err
    }

```

```

    }
    if v == nil {
        return nil, OutcomeMiss, nil
    }
    return v, OutcomeHit, nil
}

func (c *Client) Set(ctx context.Context, key string, value
    []byte, ttl time.Duration) error {
    return c.inner.Set(ctx, key, value, ttl)
}

func (c *Client) Invalidate(ctx context.Context, key string)
    error {
    return c.inner.Delete(ctx, key)
}

```



Step 2: Unit test for Key normalisation

lava-api-go/internal/cache/cache_test.go:

```

package cache

import "testing"

func TestKeyDeterministic(t *testing.T) {
    a := Key("GET", "/forum/{id}",
        map[string]string{"id": "42"},
        map[string][]string{"q": {"foo"}, "page": {"1"}},
        "abc",
    )
    b := Key("GET", "/forum/{id}",
        map[string]string{"id": "42"},
        map[string][]string{"page": {"1"}, "q": {"foo"}},
        "abc",
    )
    if a != b {
        t.Fatalf("Key not deterministic across query-key order:
            \n %s\n %s", a, b)
    }
}

func TestKeyDistinguishesPathVars(t *testing.T) {
    a := Key("GET", "/topic/{id}", map[string]string{"id": "1"},
        nil, "")
    b := Key("GET", "/topic/{id}", map[string]string{"id": "2"},
        nil, "")
    if a == b { t.Fatal("Key collided across distinct id
        values") }
}

func TestKeyDistinguishesAuthRealm(t *testing.T) {

```

```

a := Key("GET", "/favorites", nil, nil, "user-A")
b := Key("GET", "/favorites", nil, nil, "user-B")
if a == b { t.Fatal("Key collided across distinct auth
    realms") }
}

func TestKeyAnonDefault(t *testing.T) {
a := Key("GET", "/x", nil, nil, "")
b := Key("GET", "/x", nil, nil, "anon")
if a != b { t.Fatal("Key for empty realm must match key for
    explicit 'anon'") }
}

```



Step 3: integration_test.go — same Postgres in podman pattern as submodules/cache

```

package cache_test

import (
    "context"
    "os"
    "testing"
    "time"

    "digital.vasic.lava.apigo/internal/cache"
    pgcache "digital.vasic.cache/pkg/postgres"
)

func mustClient(t *testing.T) (*cache.Client, func()) {
t.Helper()
url := os.Getenv("POSTGRES_TEST_URL")
if url == "" {
t.Skip("POSTGRES_TEST_URL not set; run scripts/run-test-
pg.sh")
}
inner, err := pgcache.ConnectFromURL(context.Background(),
    &pgcache.Config{
        URL: url, SchemaName: "lava_api_test", TableName:
        "response_cache_t", GCInterval: 0,
    })
if err != nil { t.Fatalf("ConnectFromURL: %v", err) }
if err := inner.CreateSchema(context.Background()); err !=
    nil {
    _ = inner.Close()
    t.Fatalf("CreateSchema: %v", err)
}
return cache.New(inner), func() {
    _, _ = inner.Underlying().Exec(context.Background(),
        `DROP SCHEMA IF EXISTS lava_api_test CASCADE`)
    _ = inner.Close()
}
}

```



```

func TestSetGetReturnsHit(t *testing.T) {
    c, cleanup := mustClient(t); defer cleanup()
    ctx := context.Background()
    if err := c.Set(ctx, "k", []byte("v"), time.Minute); err !=
        nil { t.Fatalf("Set: %v", err) }
    got, outcome, err := c.Get(ctx, "k")
    if err != nil { t.Fatalf("Get: %v", err) }
    if outcome != cache.OutcomeHit { t.Errorf("outcome=%q want
        hit", outcome) }
    if string(got) != "v" { t.Errorf("got=%q want v",
        string(got)) }
}

func TestGetMissReturnsMiss(t *testing.T) {
    c, cleanup := mustClient(t); defer cleanup()
    got, outcome, err := c.Get(context.Background(), "absent")
    if err != nil { t.Fatalf("Get: %v", err) }
    if outcome != cache.OutcomeMiss { t.Errorf("outcome=%q want
        miss", outcome) }
    if got != nil { t.Errorf("got=%q want nil", string(got)) }
}

func TestInvalidate(t *testing.T) {
    c, cleanup := mustClient(t); defer cleanup()
    ctx := context.Background()
    _ = c.Set(ctx, "k", []byte("v"), time.Minute)
    if err := c.Invalidate(ctx, "k"); err != nil {
        t.Fatalf("Invalidate: %v", err) }
    _, outcome, _ := c.Get(ctx, "k")
    if outcome != cache.OutcomeMiss { t.Errorf("outcome after
        Invalidate=%q want miss", outcome) }
}

```



Step 4: Run unit tests, expect PASS

```
( cd lava-api-go && go test -race -count=1 ./internal/cache/... )
```



Step 5: Run integration tests via the launcher

```
( cd lava-api-go && ./scripts/run-test-pg.sh )
```

Expected: PASS for the three integration tests.



Step 6: Falsifiability rehearsal

Mutate OutcomeHit to OutcomeMiss in cache.go's Get when v != nil. Re-run integration. Expected: TestSetGetReturnsHit fails. Revert.



Step 7: Commit

```
git add lava-api-go/internal/cache/
git commit -m "lava-api-go: internal/cache – facade over
    submodules/cache/pkg/postgres exposing Key (deterministic
    SHA-256 of route + path-vars + sorted-query + auth-realm
    per spec §6) and Outcome (hit/miss/bypass/invalidate) for
    metrics. Unit tests pin Key determinism + collision
    properties; integration tests against podman Postgres
    exercise full Set/Get/Invalidate cycle. Falsifiability:
    swapping OutcomeHit↔OutcomeMiss in Get fails
    TestSetGetReturnsHit."
for r in github gitflic gitlab gitverse; do git push "$r" master;
done
```

Phase 5 — Auth pass-through + rate limit + login throttle

These are Lava-domain (auth pass-through with H1 hashing is somewhat product-specific) and thin glue (rate limit). Both land together because the rate limit middleware uses the auth realm hash as one keying axis on routes that vary by user.

Task 5.1: internal/auth — passthrough header forwarding + audit hashing

Files:

- Create: lava-api-go/internal/auth/passthrough.go
- Create: lava-api-go/internal/auth/passthrough_test.go



Step 1: Write passthrough.go

```
// Package auth implements the Sixth-Law-bound pass-through auth
// model:
// the Auth-Token header from the client is forwarded verbatim to
// upstream
// rutracker.org as a session cookie, and is SHA-256-hashed into
// the
// auth_realm_hash column of request_audit. The plaintext token is
// never
// persisted, never logged, never inserted into a span attribute.
package auth

import (
    "crypto/sha256"
    "encoding/hex"
    "net/http"
```

```

    "github.com/gin-gonic/gin"
)

// HeaderName is the wire-level header read from the client and
// forwarded
// to upstream as a Cookie. Name kept identical to the Ktor proxy.
const HeaderName = "Auth-Token"

// RealmHash returns the SHA-256 hex of the Auth-Token, or empty
// string if
// no token was sent. Empty string = anonymous; do NOT mistake it
// for a
// hash of the empty string.
func RealmHash(r *http.Request) string {
    tok := r.Header.Get(HeaderName)
    if tok == "" { return "" }
    sum := sha256.Sum256([]byte(tok))
    return hex.EncodeToString(sum[:])
}

// UpstreamCookie translates the Auth-Token header into the Cookie
// value
// the rutracker.org session model expects. Centralized here so a
// future
// rutracker schema change is a one-file fix.
func UpstreamCookie(r *http.Request) string {
    tok := r.Header.Get(HeaderName)
    if tok == "" { return "" }
    // Existing Kotlin proxy uses bb_session – see proxy/src/main/
    // kotlin/.../api/HttpClientFactory.kt
    return "bb_session=" + tok
}

// GinMiddleware computes the realm hash once per request and
// stores it
// on the gin.Context for handlers and the audit writer to
// consume.
func GinMiddleware() gin.HandlerFunc {
    return func(c *gin.Context) {
        c.Set("auth_realm_hash", RealmHash(c.Request))
        c.Next()
    }
}

// HashFromContext returns the realm hash set by GinMiddleware, or
// "",
func HashFromContext(c *gin.Context) string {
    v, _ := c.Get("auth_realm_hash")
    s, _ := v.(string)
    return s
}

```



Step 2: Test for RealmHash determinism + emptiness

```
package auth

import (
    "net/http"
    "net/http/httptest"
    "testing"

    "github.com/gin-gonic/gin"
)

func TestRealmHashEmptyForMissingHeader(t *testing.T) {
    req := httptest.NewRequest("GET", "/", nil)
    if got := RealmHash(req); got != "" {
        t.Errorf("got=%q want empty", got)
    }
}

func TestRealmHashStable(t *testing.T) {
    r := func() *http.Request {
        req := httptest.NewRequest("GET", "/", nil)
        req.Header.Set(HeaderName, "secret-token")
        return req
    }
    a := RealmHash(r())
    b := RealmHash(r())
    if a != b || a == "" || len(a) != 64 {
        t.Fatalf("hash unstable or wrong length: a=%q b=%q", a, b)
    }
}

func TestUpstreamCookieFormatsCorrectly(t *testing.T) {
    req := httptest.NewRequest("GET", "/", nil)
    req.Header.Set(HeaderName, "abcd")
    if c := UpstreamCookie(req); c != "bb_session=abcd" {
        t.Errorf("got=%q want bb_session=abcd", c)
    }
}

func TestGinMiddlewareSetsContextValue(t *testing.T) {
    gin.SetMode(gin.TestMode)
    r := gin.New()
    r.Use(GinMiddleware())
    var observed string
    r.GET("/", func(c *gin.Context) { observed =
        HashFromContext(c) })
    req := httptest.NewRequest("GET", "/", nil)
    req.Header.Set(HeaderName, "x")
    w := httptest.NewRecorder()
    r.ServeHTTP(w, req)
    if observed == "" || len(observed) != 64 {
```

```

        t.Fatalf("middleware did not set realm hash: %q",
            observed)
    }
}

```



Step 3: Run tests

```
( cd lava-api-go && go test -race -count=1 ./internal/auth/... )
```



Step 4: Falsifiability rehearsal

Mutate RealmHash to always return "FIXED". Run TestRealmHashStable — passes (still stable). Run TestGinMiddlewareSetsContextValue — also passes (length is wrong though). The 64-char-length check catches it: if len(observed) != 64 fires. Revert.



Step 5: Commit

```

git add lava-api-go/internal/auth/
git commit -m "lava-api-go: internal/auth - A2 pass-through + H1
    plain SHA-256 audit hashing. RealmHash returns
    hex(sha256(Auth-Token)) or empty for anon; UpstreamCookie
    formats the bb_session= cookie value the rutracker
    upstream expects (centralised so an upstream schema change
    is one-file). GinMiddleware caches the hash on the request
    context. Falsifiability: forcing RealmHash to a fixed
    string fails the 64-char-length assertion in
    TestGinMiddlewareSetsContextValue."
for r in github gitflic gitlab gitverse; do git push "$r" master;
done

```

Task 5.2: internal/ratelimit — sliding-window middleware glue

Files:

- Create: lava-api-go/internal/ratelimit/ratelimit.go
- Create: lava-api-go/internal/ratelimit/ratelimit_test.go



Step 1: Implement the package

```

// Package ratelimit configures submodules/ratelimiter for the
// four route
// classes lava-api-go uses (read | write | login | download),
// keyed by
// (client_ip, route_class). Per spec §9, defaults are
// placeholders until
// load testing pins them; tunable via env vars.

```

```

package ratelimit

import (
    "net/http"
    "time"

    rl "digital.vasic.ratelimiter/pkg/limiter"
    rlmem "digital.vasic.ratelimiter/pkg/memory"
    "github.com/gin-gonic/gin"
)

// RouteClass enumerates the rate-limit categories.
type RouteClass string

const (
    ClassRead      RouteClass = "read"
    ClassWrite     RouteClass = "write"
    ClassLogin     RouteClass = "login"
    ClassDownload RouteClass = "download"
)

// Config holds per-class limits. Zero rate disables limiting for
// that class.
type Config struct {
    ReadRPM      int
    WriteRPM     int
    LoginRPM     int
    DownloadRPM  int
}

// DefaultConfig – placeholders per spec §9, tunable via env.
func DefaultConfig() Config {
    return Config{ReadRPM: 60, WriteRPM: 10, LoginRPM: 5,
        DownloadRPM: 10}
}

// Limiter holds one limiter per class.
type Limiter struct {
    limiters map[RouteClass]rl.Limiter
}

func New(cfg Config) *Limiter {
    mk := func(rpm int) rl.Limiter {
        if rpm <= 0 { return nil }
        return rlmem.New(&rl.Config{Rate: rpm, Window:
            time.Minute})
    }
    return &Limiter{limiters: map[RouteClass]rl.Limiter{
        ClassRead:      mk(cfg.ReadRPM),
        ClassWrite:     mk(cfg.WriteRPM),
        ClassLogin:     mk(cfg.LoginRPM),
        ClassDownload:  mk(cfg.DownloadRPM),
    }}
}

```

```

}

// Middleware returns a Gin middleware for the given route class.
// Calls to Allow are keyed by client IP.
func (l *Limiter) Middleware(class RouteClass) gin.HandlerFunc {
    lim := l.limiters[class]
    if lim == nil { return func(c *gin.Context) { c.Next() } }
    return func(c *gin.Context) {
        ip := c.ClientIP()
        ok, err := lim.Allow(c.Request.Context(),
            ip+": "+string(class))
        if err != nil { c.Next();
            return } // fail-open on limiter errors per RateLimiter
            convention
        if !ok.Allowed {
            c.AbortWithStatusJSON(http.StatusTooManyRequests,
                gin.H{"error": "rate limited"})
            return
        }
        c.Next()
    }
}
}

```



Step 2: Tests — assert that exceeding the rate yields 429.

```

package ratelimit

import (
    "net/http"
    "net/http/httptest"
    "testing"

    "github.com/gin-gonic/gin"
)

func TestMiddlewareBlocksAfterLimit(t *testing.T) {
    gin.SetMode(gin.TestMode)
    r := gin.New()
    l := New(Config{ReadRPM: 2}) // very low limit
    r.GET("/x", l.Middleware(ClassRead), func(c *gin.Context) {
        c.String(http.StatusOK, "ok")
    })

    for i := 0; i < 2; i++ {
        w := httptest.NewRecorder()
        req := httptest.NewRequest("GET", "/x", nil)
        req.RemoteAddr = "1.2.3.4:1234"
        r.ServeHTTP(w, req)
        if w.Code != http.StatusOK {
            t.Fatalf("request %d: code=%d want 200", i, w.Code)
        }
    }
}

```

```

w := httptest.NewRecorder()
req := httptest.NewRequest("GET", "/x", nil)
req.RemoteAddr = "1.2.3.4:1234"
r.ServeHTTP(w, req)
if w.Code != http.StatusTooManyRequests {
    t.Fatalf("3rd request: code=%d want 429", w.Code)
}
}

```



Step 3: Run tests

```

( cd lava-api-go && go test -race -count=1 ./internal/
  ratelimit/... )

```



Step 4: Falsifiability rehearsal

Change `if !ok.Allowed`
`{ c.AbortWithStatusJSON(http.StatusTooManyRequests, ...) }` to `if false { ... }`. Re-run; expect FAIL because the 3rd request returns 200. Revert.



Step 5: Commit

```

git add lava-api-go/internal/ratelimit/
git commit -m "lava-api-go: internal/ratelimit - submodules/
  ratelimiter sliding-window glue per route class. Defaults
  from spec §9 (60/10/5/10 RPM read/write/login/download),
  env-tunable. Test pins 429 behavior beyond limit.
  Falsifiability: nil'ing the abort branch lets the 3rd
  request through and fails TestMiddlewareBlocksAfterLimit."
for r in github gitflic gitlab gitverse; do git push "$r" master;
done

```

Phase 6 — rutracker scraper (Lava-domain)

This phase ports the Kotlin Jsoup-based scrapers in `core/network/rutracker/...` to Go. It's the largest single chunk of Lava-domain code. The strategy: **golden-fixture tests against captured rutracker HTML** so the Go scraper can be developed offline and the parity test (Phase 10) is the live double-check.

Task 6.1: HTTP client wrapped with submodules/recovery circuit breaker

Files:

- Create: lava-api-go/internal/rutacker/client.go
- Create: lava-api-go/internal/rutacker/client_test.go



Step 1: client.go — net/http client with breaker

```
// Package rutacker is the Lava-domain rutacker.org scraper. It  
    wraps an  
// HTTP client with a circuit breaker (submodules/recovery),  
    forwards the  
// auth cookie produced by internal/auth.UpstreamCookie, and  
    exposes typed  
// helpers each route handler invokes.
```

```
package rutacker
```

```
import (  
    "context"  
    "errors"  
    "fmt"  
    "io"  
    "net/http"  
    "time"  
  
    "digital.vasic.recovery/pkg/breaker"  
)
```

```
type Client struct {  
    base    string  
    http    *http.Client  
    breaker *breaker.CircuitBreaker  
}
```

```
func NewClient(base string) *Client {  
    return &Client{  
        base: base,  
        http: &http.Client{Timeout: 30 * time.Second},  
        breaker: breaker.NewCircuitBreaker("rutacker",  
            breaker.Config{  
                MaxFailures: 5,  
                Window:      30 * time.Second,  
                OpenWindow: 10 * time.Second,  
            }),  
    }  
}
```

```
// Fetch performs a GET against base+path with the given cookie  
    value
```

```

// (may be empty for anonymous requests). It returns the response
// body
// bytes and the upstream status code; transient errors trip the
// breaker.
func (c *Client) Fetch(ctx context.Context, path, cookie string)
    ([]byte, int, error) {
    var body []byte
    var status int
    err := c.breaker.Execute(func() error {
        req, err := http.NewRequestWithContext(ctx, "GET",
            c.base+path, nil)
        if err != nil { return err }
        if cookie != "" { req.Header.Set("Cookie", cookie) }
        resp, err := c.http.Do(req)
        if err != nil { return err }
        defer resp.Body.Close()
        b, err := io.ReadAll(resp.Body)
        if err != nil { return err }
        body = b
        status = resp.StatusCode
        // Treat 5xx as breaker-relevant errors.
        if resp.StatusCode >= 500 {
            return fmt.Errorf("rutracker upstream %d",
                resp.StatusCode)
        }
        return nil
    })
    if err != nil { return nil, 0, err }
    return body, status, nil
}

// ErrCircuitOpen is returned when the breaker is in OPEN state
// and the
// request was not attempted.
var ErrCircuitOpen = errors.New("rutracker: circuit breaker OPEN")

```



Step 2: Test using httptest.Server as a stand-in upstream

```

package rutracker

import (
    "context"
    "net/http"
    "net/http/httptest"
    "testing"
)

func TestFetchReturnsBodyAndStatus(t *testing.T) {
    srv := httptest.NewServer(http.HandlerFunc(func(w
        http.ResponseWriter, r *http.Request) {
        w.Header().Set("Content-Type", "text/html")
        w.WriteHeader(http.StatusOK)
        w.Write([]byte("<html>hi</html>"))
    })
}

```

```

    )))
    defer srv.Close()
    c := NewClient(srv.URL)
    body, status, err := c.Fetch(context.Background(), "/", "")
    if err != nil { t.Fatalf("Fetch: %v", err) }
    if status != 200 { t.Errorf("status=%d want 200", status) }
    if string(body) != "<html>hi</html>" { t.Errorf("body=%q",
        string(body)) }
}

func TestFetchTrips5xxIntoBreakerError(t *testing.T) {
    srv := httptest.NewServer(http.HandlerFunc(func(w
        http.ResponseWriter, r *http.Request) {
            w.WriteHeader(http.StatusInternalServerError)
        }))
    defer srv.Close()
    c := NewClient(srv.URL)
    _, _, err := c.Fetch(context.Background(), "/", "")
    if err == nil { t.Fatal("expected error for 500 response") }
}

```



Step 3: Run, verify, commit (no falsifiability rehearsal needed for httptest-based plumbing — tests already exercise real wire)

Task 6.2 – 6.7: Per-section scrapers (forum, search, topic, comments, torrent, favorites, login, captcha)

Implementation note: the Kotlin scrapers in `core/network/rutracker/.../domain/` use Jsoup to walk the HTML DOM. Go has two equivalents: `golang.org/x/net/html` (stdlib, awkward) or `github.com/PuerkitoBio/goquery` (Jsoup-ish API). Use **goquery** for direct portability. Each file below is a 1:1 port of the corresponding Kotlin file.

For each of the seven sections (forum, search, topic, comments, torrent, favorites, login, captcha):

Files (per section):

- Create: `lava-api-go/internal/rutracker/<section>.go`
- Create: `lava-api-go/internal/rutracker/<section>_test.go`
- Create: `lava-api-go/internal/rutracker/testdata/<section>/*.html` (fixtures)

Per-section steps (uniform pattern):



Step 1: Read the corresponding Kotlin file in `core/network/rutracker/src/main/kotlin/lava/network/domain/<Section>UseCase.kt` and the parser at `lava/network/domain/Parse<Section>...UseCase.kt`. List every CSS selector and every output field.



Step 2: Capture a real `rutracker.org` page. With the Lava Ktor proxy running locally, hit the corresponding endpoint and record the upstream response body to `testdata/<section>/<scenario>.html`. (Capture at least: empty result, single result, paginated with multiple results, malformed/edge case if reachable.)



Step 3: Define the Go struct for the parsed result. Match the field names to the OpenAPI schema authored in Task 2.1.



Step 4: Write a test that loads each fixture and asserts the expected struct.



Step 5: Implement the parser using `goquery`. Run the test until it passes.



Step 6: Add a `Fuzz<Section>Parse` test that feeds adversarial bytes (truncated HTML, invalid UTF-8, deeply nested tags) and asserts no panic, no goroutine leak.



Step 7: Falsifiability rehearsal — pick one selector, mutate it (e.g. `td.t-name` → `td.t-wrong`), re-run, observe the relevant test fail with a clear "expected N items, got 0" message. Revert.



Step 8: Commit per section. Push to all four upstreams.

This pattern produces 7 commits. The complete commit set is:

Commit	Section	Routes consumed by
6.2	forum	<code>/forum, /forum/{id}</code>
6.3	search	<code>/search</code>
6.4	topic	<code>/topic/{id}, /topic2/{id}, /comments/{id}</code>
6.5	comments-add	<code>POST /comments/{id}/add</code>
6.6	torrent	<code>/torrent/{id}, /download/{id}</code>
6.7	favorites	

Commit	Section	Routes consumed by
		/favorites, POST /favorites/{add,remove}/{id}
6.8	login + captcha	POST /login, GET /captcha/{path}

Phase 7 — Route handlers (the 13 routes)

For each of the 13 + 3 endpoints (the table in spec §6), wire: cache lookup → on miss, scraper call → cache store → response. Writes additionally invalidate cache keys.

Task pattern (one task per route group, ~7 tasks total)

For each route group (matching the rutracker scraper sections), follow this pattern:

Files (per group):

- Create: lava-api-go/internal/handlers/<group>.go
- Create: lava-api-go/internal/handlers/<group>_test.go
- Modify: lava-api-go/internal/handlers/handlers.go (the route-registration entry point)



Step 1: Define a handler struct that holds the dependencies the route needs: Cache, Limiter, Rutracker (a section-specific scraper interface). Constructor NewXxxHandler(cache, limiter, rutracker).



Step 2: Write a contract test: feed a request, assert the response matches the OpenAPI schema (use the typed client from internal/gen/client/). Read fixtures from the rutracker scraper's testdata.



Step 3: Implement the handler. The structure is:

```
func (h *ForumHandler) GetForum(c *gin.Context) {
    realm := auth.HashFromContext(c)
    key := cache.Key("GET", "/forum", nil, c.Request.URL.Query(),
        realm)
    if cached, outcome, err := h.Cache.Get(c.Request.Context(),
        key); err == nil && outcome == cache.OutcomeHit {
        c.Data(http.StatusOK, "application/json", cached)
        return
    }
}
```

```
// miss → scraper
upstreamBody, status, err :=
    h.Rutracker.GetForum(c.Request.Context(),
        auth.UpstreamCookie(c.Request))
if err != nil {
    c.JSON(http.StatusBadGateway, gin.H{"error":
        err.Error()}); return
}
parsed, err := h.Rutracker.ParseForum(upstreamBody)
if err != nil {
    c.JSON(http.StatusBadGateway, gin.H{"error":
        err.Error()}); return
}
body, _ := json.Marshal(parsed)
_ = h.Cache.Set(c.Request.Context(), key, body, 1*time.Hour)
c.Data(status, "application/json", body)
}
```



Step 4: Run contract test, expect PASS.



Step 5: Falsifiability rehearsal (mandatory). Mutate the response — e.g. drop a JSON field, or return wrong status. Confirm the contract test catches it. Revert.



Step 6: Register the handler in handlers.go's Register(router *gin.Engine, deps *Deps) function.



Step 7: Commit per group.

The seven handler-group commits: forum, search, topic, comments, torrent, favorites, login+captcha+index. Mirror the Phase 6 scraper commits.

Phase 8 — mDNS advertisement + healthprobe binary

Task 8.1: internal/discovery/mdns.go

Files:

- Create: lava-api-go/internal/discovery/mdns.go
- Create: lava-api-go/internal/discovery/mdns_test.go



Step 1: Implement using submodules/mdns

package discovery

```
import (
    "context"

    "digital.vasic.mdns/pkg/service"

    "digital.vasic.lava.apigo/internal/version"
)

func Announce(ctx context.Context, instance, serviceType string,
    port int) (*service.Service, error) {
    return service.Announce(service.Announcement{
        Name:      instance,
        ServiceType: serviceType,
        Port:      port,
        TXT: map[string]string{
            "engine":      "go",
            "version":     version.Name,
            "protocols":   "h3,h2",
            "compression": "br,gzip",
            "tls":         "required",
            "path":        "/",
        },
    })
}
```



Step 2: Integration test (skip if multicast unavailable, similar to submodules/mdns's pattern).



Step 3: Falsifiability rehearsal — drop the engine=go TXT key in the production code, observe test failure. Revert.



Step 4: Commit + push.

Task 8.2: cmd/healthprobe/main.go

Files:

- Create: lava-api-go/cmd/healthprobe/main.go



Step 1: 30-line probe binary

```

// healthprobe is a CLI used by the container's HEALTHCHECK
// directive to
// verify the lava-api-go service is reachable over HTTP/3. Exits
// 0 on a
// 200 response; non-zero otherwise. Baked into the runtime image.
package main

import (
    "crypto/tls"
    "flag"
    "fmt"
    "net/http"
    "os"
    "time"

    "github.com/quic-go/quic-go/http3"
)

func main() {
    url := flag.String("url", "https://localhost:8443/health",
        "URL to probe")
    insecure := flag.Bool("insecure", true,
        "skip TLS verification (self-signed cert)")
    flag.Parse()

    cli := &http.Client{Transport: &http3.Transport{
        TLSClientConfig: &tls.Config{InsecureSkipVerify:
            *insecure, NextProtos: []string{"h3"}},
        }, Timeout: 2 * time.Second}
    resp, err := cli.Get(*url)
    if err != nil { fmt.Fprintln(os.Stderr, "probe error:", err);
        os.Exit(1) }
    defer resp.Body.Close()
    if resp.StatusCode != 200 {
        fmt.Fprintln(os.Stderr, "probe status:",
            resp.StatusCode); os.Exit(1)
    }
    fmt.Println("ok")
}

```



Step 2: Build and smoke-test.



Step 3: Commit + push.

Phase 9 — Main entry point + Dockerfile

Task 9.1: cmd/lava-api-go/main.go — wire all the pieces

Files:

- Create: lava-api-go/cmd/lava-api-go/main.go



Step 1: The main function: load config → init logger → init metrics → init tracer → connect Postgres → init cache → init rate limiter → init rutracker scraper → register handlers → start mDNS announcement → start server → wait for SIGINT/SIGTERM → graceful shutdown.



Step 2: Smoke test: `go run ./cmd/lava-api-go --help` and confirm it parses.



Step 3: Commit + push.

Task 9.2: docker/Dockerfile multi-stage

Files:

- Create: lava-api-go/docker/Dockerfile



Step 1: Multi-stage Dockerfile

```
# syntax=docker/dockerfile:1.7
```

```
# --- build stage ---
```

```
FROM golang:1.24-alpine AS build
```

```
WORKDIR /src
```

```
COPY . .
```

```
RUN cd /src/lava-api-go && \
```

```
    go build -trimpath -ldflags='-s -w' -o /out/lava-api-go ./cmd/  
        lava-api-go && \
```

```
    go build -trimpath -ldflags='-s -w' -o /out/healthprobe ./cmd/  
        healthprobe
```

```
# --- migrate stage (one-shot lava-migrate compose service) ---
```

```
FROM golang:1.24-alpine AS migrate
```

```
WORKDIR /src
```

```
COPY . .
```

```
RUN cd /src/lava-api-go && go install -tags 'postgres' github.com/  
    golang-migrate/migrate/v4/cmd/migrate@latest
```

```
ENTRYPOINT ["/go/bin/migrate"]
```

```

CMD ["-path", "/src/lava-api-go/migrations", "-database",
      "$LAVA_API_PG_URL", "up"]

# --- runtime stage ---
FROM gcr.io/distroless/static-debian12:nonroot AS runtime
COPY --from=build /out/lava-api-go /usr/local/bin/lava-api-go
COPY --from=build /out/healthprobe /usr/local/bin/healthprobe
EXPOSE 8443/udp 8443/tcp
HEALTHCHECK --interval=10s --retries=6 CMD ["/usr/local/bin/
      healthprobe"]
ENTRYPOINT ["/usr/local/bin/lava-api-go"]

```



Step 2: Build the image: make image from lava-api-go/.



Step 3: Smoke run: podman run --rm -d --name lavatest -p 8443:8443 ... lava-api-go:dev then curl --http3 -k https://localhost:8443/health. Expected: {"status":"alive"}.



Step 4: Commit + push.

Phase 10 — Test infrastructure (e2e + parity + load + contract)

Task 10.1: tests/contract — OpenAPI golden fixtures

For every OpenAPI route, create a <routeName>.golden.json fixture. Tests use the typed client from internal/gen/client/ to round-trip request/response and compare to the fixture.



Step 1: For each of 16 endpoints, capture a representative response from the running Go API and pin it to tests/fixtures/contract/<route>.golden.json.



Step 2: Implement tests/contract/contract_test.go that loads each fixture and asserts equality.



Step 3: Falsifiability rehearsal: change one byte in a handler response, observe the contract test catch it.



Step 4: Commit + push.

Task 10.2: tests/e2e — real container, real HTTP/3 client



Step 1: `tests/e2e/e2e_test.go` brings up a real `lava-api-go` container via `podman` + a `Postgres` container, runs the migration, then exercises every route over real `HTTP/3`.



Step 2: Falsifiability rehearsal.



Step 3: Commit + push.

Task 10.3: `tests/parity` — Sixth Law load-bearing gate (cross-backend)



Step 1: `tests/parity/parity_test.go` brings up BOTH the `Ktor` proxy (existing image) AND the `Go` API. Uses the typed client from `internal/gen/client/` to send identical requests to both. Asserts byte-equivalence on body, status code, and the user-visible header subset.



Step 2: Populate `tests/fixtures/parity/` with a comprehensive case matrix per spec §11.1: all 16 endpoints × {anon, authenticated} × {empty/tiny/medium body for POSTs}.



Step 3: Falsifiability rehearsal — three different mutations: (a) corrupt a response body in the `Go` API, (b) reorder `JSON` object keys, (c) drop a header. Each must produce a clear, targeted failure.



Step 4: Commit + push.

Task 10.4: `tests/load` — `k6` quick + soak



Step 1: `tests/load/k6-quick.js` — 60s smoke load with thresholds (p99 < 200ms on cached hits).



Step 2: `tests/load/k6-soak.js` — 30min soak (manual run, not in default ci).



Step 3: `scripts/load-quick.sh` invokes `k6-quick.js` against a running container and asserts threshold pass.



Step 4: Commit + push.

Phase 11 — Container topology: profiles compose + observability + Lava CLI

Task 11.1: Rewrite root docker-compose.yml with profiles

Files:

- Modify: docker-compose.yml (full rewrite)



Step 1: Replace the existing single-service compose with the multi-profile shape from spec §12.1 (full content quoted in spec — paste verbatim).



Step 2: Smoke test each profile:

docker compose --profile api-go up -d, --profile legacy, etc.



Step 3: Commit + push.

Task 11.2: docker/observability config files

Files:

- Create: lava-api-go/docker/observability/{prometheus.yml, loki-config.yml, promtail-config.yml, tempo.yml}
- Create: lava-api-go/docker/observability/grafana/provisioning/{datasources, dashboards}/...
- Create: lava-api-go/docker/observability/grafana/dashboards/lava-api.json



Step 1: Author each config file. Standard documented configs per the Grafana docs; consume the metric/log/trace endpoints listed in spec §10.2.



Step 2: docker compose --profile observability up -d and verify each container's health endpoint responds.



Step 3: Open <http://127.0.0.1:3000> (Grafana), confirm the lava-api dashboard loads with non-empty graphs while lava-api-go is serving traffic.



Step 4: Commit + push.

Task 11.3: Extend tools/lava-containers/ with profile orchestration

Files:

- Modify: tools/lava-containers/cmd/lava-containers/main.go (add --profile and --observability flags)
- Modify: tools/lava-containers/internal/proxy/proxy.go (refactor to a profile orchestrator)
- Add: tools/lava-containers/internal/orchestrator/orchestrator.go (new — knows about profiles, delegates runtime to submodules/containers)
- Modify: tools/lava-containers/go.mod (add replace digital.vasic.containers => ../../submodules/containers)



Step 1: Rewrite the CLI flag parser to accept --profile=api-go|legacy|both and --with-observability, --dev-docs.



Step 2: Replace the direct compose -f docker-compose.yml up call with invocations of submodules/containers/pkg/compose passing the right -profile flags.



Step 3: Smoke test each combination via ./bin/lava-containers -cmd=start --profile=api-go.



Step 4: Commit + push.

Phase 12 — Lifecycle scripts: start/stop, build_and_release, tag.sh

Task 12.1: Update start.sh / stop.sh

Files:

- Modify: start.sh — add --legacy, --both, --with-observability, --dev-docs flags
- Modify: stop.sh — tear down all running profiles



Step 1: Implement flag parsing, translate to tools/lava-containers/bin/lava-containers -cmd=start --profile=... invocations.



Step 2: Smoke test each combination.



Step 3: Commit + push.

Task 12.2: Extend scripts/tag.sh registry with api-go

Files:

- Modify: scripts/tag.sh
- Modify: docs/TAGGING.md



Step 1: Add api-go to SUPPORTED_APPS. Add read_apigo_version_name and read_apigo_version_code functions reading lava-api-go/internal/version/version.go (regex on the Name and Code constant declarations). Add write_apigo_versions writing them back via sed. Add the api-go case in the per-app dispatch.



Step 2: Update docs/TAGGING.md source-of-truth table with an api-go row pointing at lava-api-go/internal/version/version.go.



Step 3: scripts/tag.sh --dry-run must show Lava-API-Go-2.0.0-2000 without error.



Step 4: Commit + push.

Task 12.3: build_and_release.sh — Go build step + releases dir population

Files:

- Modify: build_and_release.sh



Step 1: Add a go build step that produces releases/{version}/api-go/lava-api-go-{version} and releases/{version}/api-go/lava-api-go-{version}.image.tar (saved container image).



Step 2: Smoke test: ./build_and_release.sh produces all three artifact directories (android-debug, android-release, proxy, api-go).



Step 3: Commit + push.

Phase 13 — Pretag verification + mutation testing + security

Task 13.1: scripts/pretag-verify.sh

Files:

- Create: lava-api-go/scripts/pretag-verify.sh
- Create: .lava-ci-evidence/.gitkeep



Step 1: Bash script that boots the full stack via `./start.sh`, executes a scripted user flow (login → search "test" → fetch first result → fetch its torrent metadata → add to favorites → remove from favorites → logout), records every HTTP status and body length to `.lava-ci-evidence/<commit>.json`. Exits 0 on success.



Step 2: Wire into `scripts/tag.sh` — refuse to tag without a matching evidence file.



Step 3: Run once, commit the evidence file.



Step 4: Commit + push.

Task 13.2: scripts/mutation.sh

Files:

- Create: lava-api-go/scripts/mutation.sh



Step 1: Wrap `go-mutesting` (or equivalent) over the `internal/` tree. Output the surviving mutants list to `mutation-report-<date>.json`.



Step 2: Smoke run; if non-trivial surviving mutants are found, file them as TODOs in `SECURITY.md` or open follow-ups.



Step 3: Commit + push.

Task 13.3: Add gosec, govulncheck, trivy to ci.sh



Step 1: Append the security steps to `lava-api-go/scripts/ci.sh` (mirror the structure used in `submodules/http3/scripts/ci.sh`).



Step 2: Resolve any HIGH/CRITICAL findings before proceeding.
Document accepted-low items in SECURITY.md.

☐

Step 3: Commit + push.

Phase 14 — Acceptance + first tag + push

Task 14.1: Run full local CI gate

☐

Step 1: `cd lava-api-go && ./scripts/ci.sh` — must complete green.

☐

Step 2: `./scripts/pretag-verify.sh` — produce `.lava-ci-evidence/<commit>.json`.

☐

Step 3: Walk through the 11 acceptance criteria in spec §18 and confirm each is met.

Task 14.2: Cut the first tag

☐

Step 1: `scripts/tag.sh --app api-go --no-push --dry-run` — verify it would create `Lava-API-Go-2.0.0-2000`.

☐

Step 2: `scripts/tag.sh --app api-go` — creates the tag and pushes to all four upstreams.

☐

Step 3: Verify the tag appears on all four remotes:

```
for r in github gitflic gitlab gitverse; do
  printf '%-10s' "$r"; git ls-remote --tags "$r" 'refs/tags/
    Lava-API-Go-*'
done
```

☐

Step 4: Verify the post-tag bump moved versions to `2.0.1 / 2001` (default `--bump patch`).

Task 14.3: Final close-out

☐

Step 1: Update `docs/superpowers/specs/2026-04-28-sp2-go-api-migration-design.md` Appendix A with implementation provenance (commit hashes that landed each phase).



Step 2: Final commit + push.



Step 3: Open SP-3 brainstorm (Android dual-backend support) when ready.

Self-review notes

Cross-checked against spec sections:

- §2 (scope): Phases 0–14 cover every in-scope item; out-of-scope items (SP-3 Android, SP-1 test bluff audit) explicitly deferred.
- §5 (decoupling boundary): every Lava-domain item (auth pass-through + audit, rutracker scraper, 13 handlers, healthprobe) has a phase; every reusable item is consumed via `replace` directive in Task 1.1's `go.mod`.
- §6 (13 routes): Phase 7 has a sub-task per route group.
- §7 (postgres data model): Phase 4 covers full DDL + migrations + cache wrapper.
- §8 (wire & mDNS): Phase 0 (Ktor TXT), Phase 8 (Go mDNS), Phase 9 (Dockerfile EXPOSE).
- §9 (auth): Phase 5 — A2 + H1 with falsifiability.
- §10 (observability): Phase 3 (app-side), Phase 11 (containers).
- §11 (test plan): Phase 10 covers types 3, 4, 5, 7; Phases 3-7 cover types 1, 2, 6 inline; Phase 13 covers types 8, mutation, pre-tag.
- §12 (container topology): Phase 11 (T1+M-A profiles), Phase 9 (healthprobe (b)).
- §13 (repo layout): Phase 1 creates the tree.
- §14 (OpenAPI D1): Phase 2.
- §15 (versioning): Task 1.4 creates `internal/version`; Task 12.2 wires `tag.sh`.
- §16 (existing files): Phase 0 (`ServiceAdvertisement.kt`, `CLAUDE.md`), Phase 11 (`docker-compose.yml`, `tools/lava-containers/`), Phase 12 (`start/stop/tag.sh/build_and_release.sh`).
- §17 (falsifiability protocol): every test-adding task has an explicit rehearsal step.
- §18 (acceptance criteria): Phase 14 walks the 11-item checklist.

No placeholders found in concrete steps. Two TBDs from the spec (login backoff schedule, rate-limit thresholds) are deliberate — listed in spec §19, deferred to load-test tuning during Phase 10.

Naming consistency check: `Limit`, `RouteClass`, `Cache`, `Outcome`, `Key` all appear with consistent signatures across `cache`, `ratelimit`, `handlers`, and `tests`.

Plan complete. Saved to docs/superpowers/plans/2026-04-28-sp2-go-api-migration.md.